考研 408 统考 2025 真题答案解析

1.下列程序段的时间复杂度是（　）。

```
int count = 0, i, j;
for(i = 1; i * i <= n; i++)
    for(j = 1; j <= i; j++)
        count++;
```

A.O(log2n)

B.O(n)

C.O(nlog2n)

D.O(n^2)

【答案】B.O(n)

复杂度计算

循环层次	循环条件	执行次数
外层循环	i*i < n	i 从 0 到 √n-1，共 √n 次
内层循环	j < i	对每个 i，执行 i 次

总执行次数计算

内层循环总执行次数为：

$$\sum_{i=0}^{\sqrt{n}-1} i = 0 + 1 + 2 + \cdots + \left(\sqrt{n} - 1\right) = \frac{\sqrt{n}\left(\sqrt{n} - 1\right)}{2}$$

时间复杂度结论

- 当 n 足够大时，主导项为 $\frac{n}{2}$
- 忽略常数因子，时间复杂度为 O(n)

【答案】B.O(n)

外层循环执行√n 次，内层循环累计执行次数约为$\frac{n}{2}$次，因此总体复杂度为线性 O(n)。

2. 已知算法 A 用于检查字符串中各类括号是否匹配，A 执行过程中使用初始为空的栈保存遇到的括号。若栈的容量是 3，则下列选项中，A 不能处理的是（ ）。

A.(a+[b+(c+d) /e]+f)+g-h

B.[a*((b+c)/(d-e)+f/g)]-h

C.[a*(b-(c-d)*e/(f+g))-h]

D.[a-(b+[c*(d+e)-f]+g+h)]

【答案】D.[a-(b+[c*(d+e)-f]+g+h)]

问题分析

本题考查的是栈容量限制下的括号匹配问题。栈的容量为 3，意味着在任何时刻，栈中存储的未匹配括号数量不能超过 3 个。

解题方法

遍历表达式中的每个字符：

- 遇到左括号：入栈
- 遇到右括号：出栈（与栈顶左括号匹配）
- 其他字符：忽略

关键是找出栈深度的最大值，如果超过 3 则不能实现。

A 选项：(a+[b+(c+d)/e]+f)+g-h ✅

括号序列：([( )])
栈变化： (→ [(→ [((→ [(→ [→ 空最大深度：3

B 选项：[a*((b+c)/(d-e)+f/g)]-h ✅

括号序列：[(( ) ( )]
栈变化： [→ [(→ [((→ [(→ [((→ [(→ 空最大深度：3

C 选项：[a*(b-(c-d)*e/(f+g))-h]

括号序列：[(( ) ( )]
栈变化： [→ [(→ [((→ [(→ [((→ [(→ 空最大深度：3

D 选项：[a-(b+[c*(d+e)-f]+g+h)]

括号序列：[([( )])
栈变化： [→ [(→ [([→ [([(→ [([→ [(→ [→ 空最大深度：4 ← 超出容量！

结果对比表

选项	表达式关键部分	最大嵌套深度	临界位置	能否处理
A	**([(**处	3	**(a+[b+(c+d)**	
B	**[((**处	3	**[a*((b+c)**	
C	**[(-**和**[(**处	3	**[a*(b-(c-d)**	
D	**[([(处**	**4**	**[a-(b+[c*(d+e)**	

【答案】D.[a-(b+[c*(d+e)-f]+g+h)]

选项 D 在解析到**[a-(b+[c*(d+e)**时需要 4 层嵌套深度，超出了栈容量 3 的限制，因此无法实现括号匹配。

3. 若二叉树的节点值均为正数，采用顺序存储方式保存在数组 R 中，用 -1 表示节点不存在，则下列数组中，不能表示一棵二叉树的是（ ）。

A.(20, 15, 40, -1, -1, 35)

B.(15, 40, 10, 18, 35, -1, -1, 12)

C.(15, 40, 10, -1, -1, -1, 12)

D.(17, 20, 35, -1, 18, 45, -1, -1, 19, 2)

【答案】D.(17, 20, 35, -1, 18, 45, -1, -1, 19, 2)

问题分析

二叉树采用顺序存储时，遵循层序遍历的索引规则：

- 根节点在索引 1（或 0）
- 对于索引为 i 的节点：
  - 左孩子在索引 2i
  - 右孩子在索引 2i+1
- **关键规则**：若某节点不存在（-1），则其左右孩子也必须不存在

各选项验证

A 选项：(20, 15, 40, -1, -1, 35) ✅正确

索引	**1**	**2**	**3**	**4**	**5**	**6**
值	20	15	40	-1	-1	35
父节点	-	1	1	2	2	3

验证：索引 4、5 为-1（节点 2 的孩子），符合规则

B 选项：(15, 40, 10, 18, 35, -1, -1, 12) ✅正确

索引	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
值	15	40	10	18	35	-1	-1	12
父节点	-	1	1	2	2	3	3	4

验证：索引 6、7 为-1（节点 3 的孩子），索引 8=12（节点 4 的孩子），符合规则

C 选项：(15, 40, 10, -1, -1, -1, 12) ✅正确

索引	**1**	**2**	**3**	**4**	**5**	**6**	**7**
值	15	40	10	-1	-1	-1	12
父节点	-	1	1	2	2	3	3

验证：索引 4、5 为-1（节点 2 的孩子），索引 6、7 分别为-1 和 12（节点 3 的孩子），符合规则 ✅

D 选项：(17, 20, 35, -1, 18, 45, -1, -1, 19, 2) ❌错误

索引	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
值	17	20	35	-1	18	45	-1	-1	19	2
父节点	-	1	1	2	2	3	3	4	4	5

问题发现：

- 索引 4 = -1（节点不存在）
- 但索引 8、9 = -1、19（节点 4 的左右孩子）
- **矛盾**：不存在的节点 4 怎么能有右孩子 19？

【答案】D.(17, 20, 35, -1, 18, 45, -1, -1, 19, 2)

选项 D 违反了二叉树顺序存储的基本规则：索引 4 的节点不存在（-1），但其右孩子位置（索引 9）却有值 19，这在逻辑上是不可能的，因为不存在的节点不能有孩子节点。

4.下列关于二叉树及森林的叙述中，正确的是（　）。

A. 完全二叉树中不存在度为 1 的节点

B. 任意一个森林都可以转换为一棵二叉树

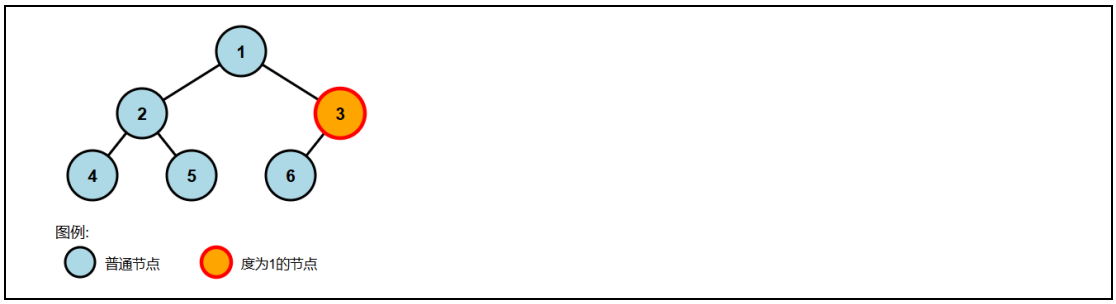
C. 二叉树的分支节点个数比叶节点个数少

D. 链式树的根中保存的是最先计算机的运算符

【答案】B. 任意一个森林都可以转换为一棵二叉树

A. 完全二叉树中不存在度为 1 的节点❌错误

错误。完全二叉树的定义允许最后一层从左到右连续填充，倒数第二层可能存在只有左孩子的节点。



节点 3 的度为 1（只有左孩子 6），但这仍是完全二叉树。

B. 任意一个森林都可以转换为一棵二叉树✅正确

正确。使用"左孩子-右兄弟"表示法，任何森林都能唯一转换为二叉树。

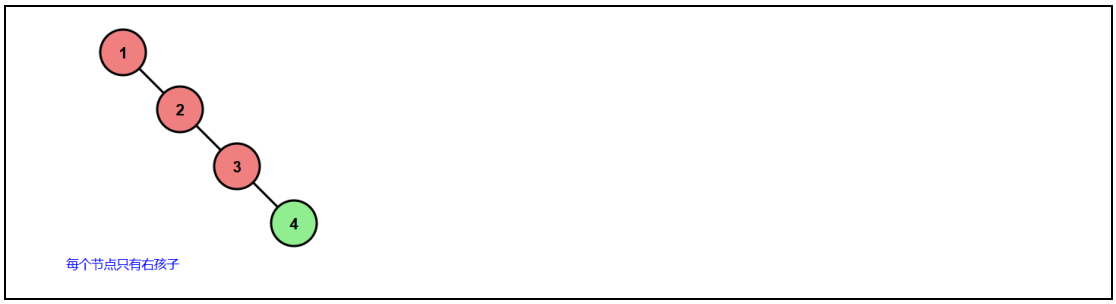
转换规则：

- 原树的第一个孩子 → 二叉树的左孩子
- 原树的兄弟节点 → 二叉树的右孩子

C. 二叉树的分支节点个数比叶节点个数少❌错误

错误。这个关系不是普遍成立的。

反例：单链式二叉树



分支节点：3 个(1,2,3)，叶节点：1 个(4)，分支节点数 > 叶节点数

D. 表达式树的根中保存的是最先计算的运算符❌错误

错误。表达式树的根保存**最后计算**的运算符。

对比总结表

选项	陈述内容	正确性	关键解释
A	完全二叉树中不存在度为 1 的节点		倒数第二层可能有只有左孩子的节点
B	**任意森林都可转换为二叉树**		**左孩子-右兄弟表示法**
C	分支节点数 < 叶节点数		单链式结构：分支节点数 > 叶节点数
D	根保存最先计算的运算符		根保存**最后**计算的运算符

【答案】B. 任意一个森林都可以转换为一棵二叉树

森林到二叉树的转换是数据结构中的经典算法，通过左孩子-右兄弟表示法可以将任意有序森林唯一地转换为一棵二叉树，这是理论上已证明的结论。

5. 设字符集 S 包含 7 个字符，各字符的出现的的频次分别为 2，3，4，6，8，10，11，为 S 中的各字符构造哈曼编码，编码长度不小于 3 的字符个数是（ ）。

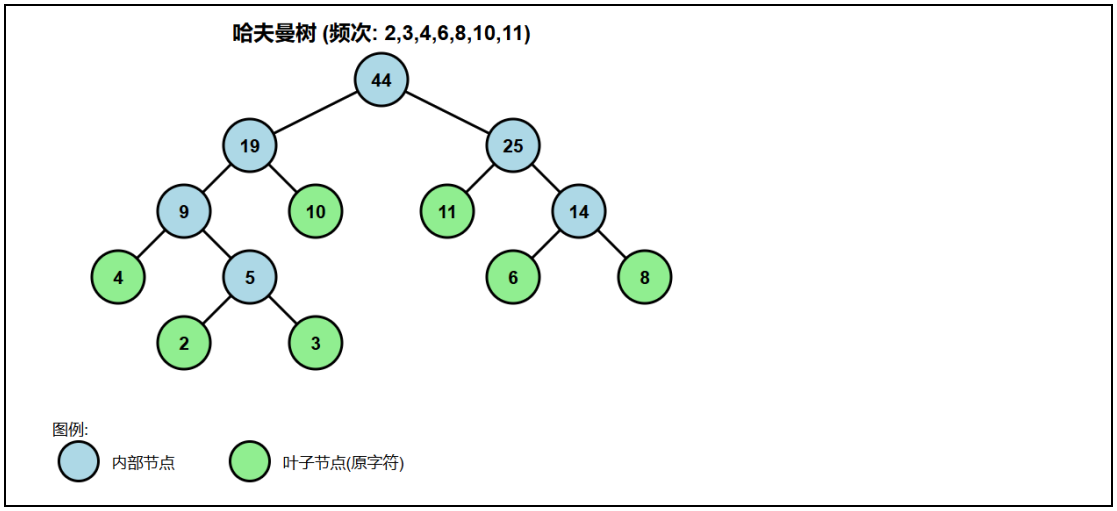
A.2 B.3 C.4 D.5

【答案】D.5

哈夫曼树构造过程

步骤	可选节点	选择合并	新节点权值	剩余节点
1	2,3,4,6,8,10,11	2+3	5	4,5,6,8,10,11
2	4,5,6,8,10,11	4+5	9	6,8,9,10,11
3	6,8,9,10,11	6+8	14	9,10,11,14
4	9,10,11,14	9+10	19	11,14,19
5	11,14,19	11+14	25	19,25
6	19,25	19+25	44	44(根)

哈夫曼树结构



编码长度分析

根据树的结构，各字符的编码长度（从根到叶子的路径长度）：

字符频次	在树中位置	编码长度	示例编码
2	左-左-左-左	**4**	0010
3	左-左-左-右	**4**	0011
4	左-左-右	**3**	000
6	右-右-左	**3**	110
8	右-右-右	**3**	111
10	左-右	2	01
11	右-左	2	10

统计结果

编码长度	字符个数	对应频次
2	2 个	10, 11
3	**3 个**	**4, 6, 8**
4	**2 个**	**2, 3**

【答案】D.5

通过构造哈夫曼树，编码长度不小于 3 的字符有 5 个，分别是频次为 2,3,4,6,8 的字符，其编码长度分别为 4,4,3,3,3。

6. 下列关于图，选项正确的是（　）。

A 有向图必定存在入度为 0 的顶点

B 有向无环图的拓扑有序序列存在且唯一

C 各顶点的度均大于等于 2 的无向图必有回路

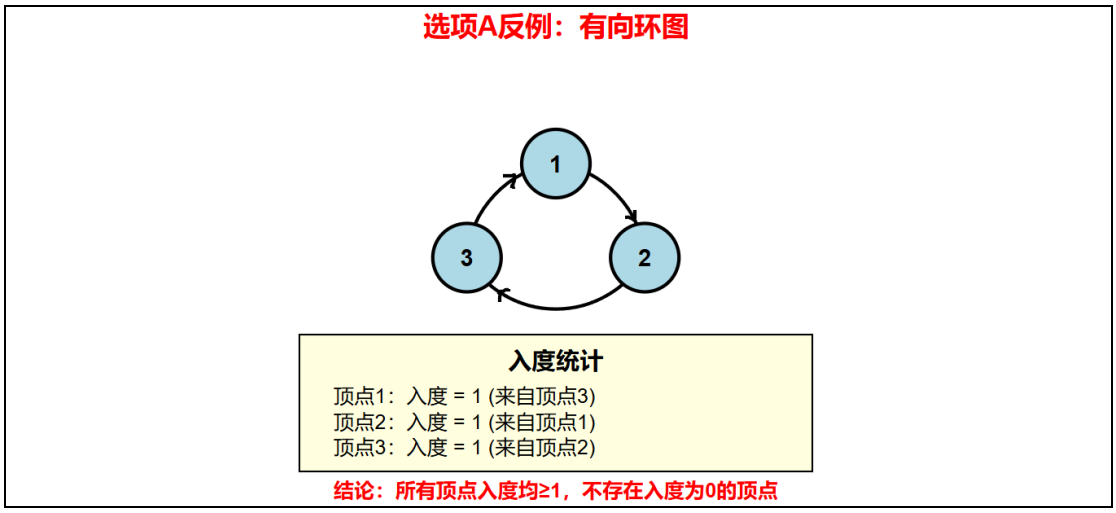
D 可用 BFS 算法求出带权图中的每一对顶点的最短路径

【答案】C 各顶点的度均大于等于 2 的无向图必有回路**A.**

有向图必定存在入度为 0 的顶错误

错误原因： 有向环图中所有顶点的入度都大于 0。

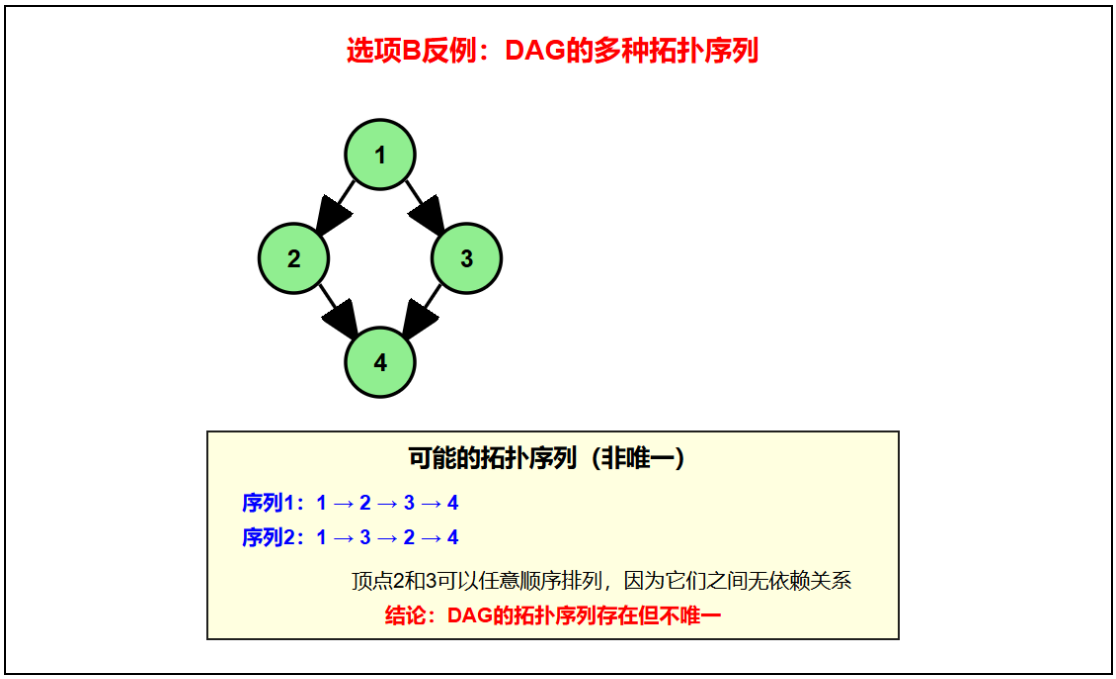
反例： 有向环图



B. 有向无环图的拓扑有序序列存在且唯一 ✗错误

错误原因： DAG 的拓扑序列存在但通常不唯一。

反例：



可能的拓扑序列：

- 1 → 2 → 3 → 4
- 1 → 3 → 2 → 4

存在多个有效的拓扑排序。

C. 各顶点的度均大于等于 2 的无向图必有回路 ✅正确

正确性证明：

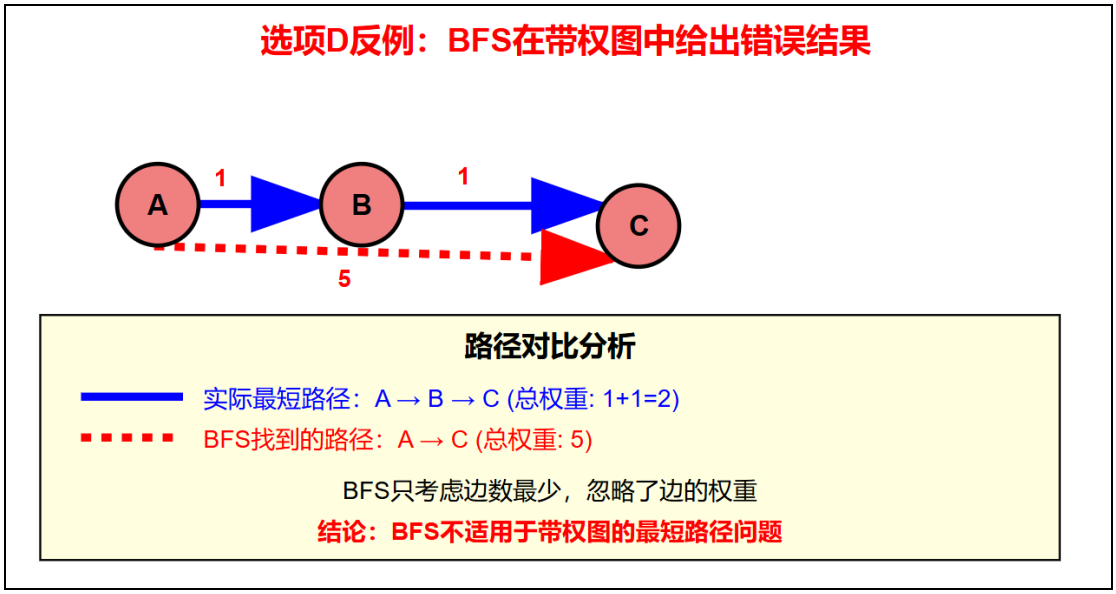
使用反证法：

1. 假设无向图 G 中所有顶点度 ≥2，且 G 无回路
2. 若 G 无回路，则 G 是森林（若干棵树的集合）
3. 在任意树中，必定存在叶子节点（度为 1 的顶点）
4. 这与"所有顶点度 ≥2"矛盾
5. 因此假设不成立，G 必有回路

D. 可用 BFS 算法求出带权图中的每一对顶点的最短路径 ❌错误

错误原因： BFS 只适用于无权图或等权图。

反例： 带权图中 BFS 可能给出错误结果



- BFS 路径：A→C（权重 5）
- 实际最短路径：A→B→C（权重 2）

带权图需使用 Dijkstra、Bellman-Ford 等专门算法。

总结对比表

选项	陈述内容	正确性	关键反例/证明
A	有向图必存在入度为 0 的顶点		有向环：所有顶点入度≥1
B	DAG 拓扑序列存在且唯一		并列节点可有多种排序
C	度≥2 的无向图必有回路		反证法：无回路则有叶子(度=1)
D	BFS 可求带权图最短路径		BFS 不考虑边权，结果错误

【答案】C 各顶点的度均大于等于 2 的无向图必有回路

关键知识点

1. **有向图性质**：强连通图中每个顶点入度≥1
2. **拓扑排序**：DAG 保证存在性，但不保证唯一性
3. **图的回路判定**：度的约束可推导回路存在性
4. **最短路径算法**：

无权图：BFS

带权图：Dijkstra（非负权）、Bellman-Ford（含负权）

所有顶点对：Floyd-Warshall

图论核心知识点总结表

知识点类别	核心概念	关键性质/算法	适用条件	时间复杂度
有向图性质	强连通图	每个顶点入度 ≥ 1	强连通分量内部	-
	弱连通图	底层无向图连通	忽略边的方向	-
拓扑排序	DAG 拓扑序列	✅ 存在性保证	无环有向图	O(V+E)
	唯一性	❌ 不保证唯一	可能有多种排序	-
	实现方法	Kahn 算法 / DFS	入度为 0 或逆后序	O(V+E)
回路判定	无向图回路	边数 ≥ 顶点数	E ≥ V 则必有回路	-
	有向图回路	DFS 检测后向边	存在后向边	O(V+E)
最短路径	无权图	**BFS 广度优先**	所有边权重相等	O(V+E)
	单源正权	**Dijkstra 算法**	非负权重	$O(V^2)$ 或 $O((V+E)\log V)$
	单源含负权	**Bellman-Ford 算法**	可检测负权回路	O(VE)
	全源最短路径	**Floyd-Warshall 算法**	所有顶点对之间	$O(V^3)$

🎯 核心记忆要点

1️⃣ 图的连通性

- **强连通** → 入度 ≥ 1（每个顶点都可达）
- **弱连通** → 忽略方向后连通

2️⃣ 拓扑排序铁律

- **DAG** → 拓扑序列**必定存在**
- **存在 ≠ 唯一** → 可能有多种排序方案

3️⃣ 最短路径算法选择

无权图 → BFS
正权图 → Dijkstra
负权图 → Bellman-Ford
全源对 → Floyd-Warshall

4️⃣ 回路判定技巧

- **无向图**：E ≥ V → 必有回路
- **有向图**：DFS 检测后向边

💡 **考试提醒**：记住算法的适用场景比记住具体实现更重要！

7. 已知查找表中有 400 个元素，查找元素概率相同。采用分块查找法确定元素所在块，且块内也采用顺序查找法，为使效率最高，每块包含元素应为（　）。

A.8

B.10

C.20

D.25

【答案】C.20

核心知识点

分块查找结构

- **索引表**：存储各块的最大关键字
- **数据块**：块间有序，块内可无序

平均查找长度公式

ASL = 索引查找长度 + 块内查找长度
ASL = (m+1)/2 + (k+1)/2

其中：

- m = 块数
- k = 每块元素个数
- n = m × k = 总元素个数

最优化求解

约束条件

- n = m × k = 400
- 目标：min ASL = (m+1)/2 + (k+1)/2

数学推导

设 f(m,k) = (m+1)/2 + (k+1)/2，约束 mk = 400

代入 m = 400/k：

f(k) = (400/k + 1)/2 + (k+1)/2
f'(k) = -200/k² + 1/2 = 0

解得：k² = 400，所以 **k = 20**

【皮皮灰】C.20

分块查找最优化定理

当块数 m = 每块元素数 k = √n 时，平均查找长度最小

一般公式

- 总元素数：n
- 最优块大小：k = √n
- 最优块数：m = √n
- 最小 ASL：√n + 1

记忆要点

1. 分块查找 = 索引查找 + 块内查找
2. 最优条件：m = k = √n
3. 本题：√400 = 20

8. 7 个不同的关键字，能够构成的不同 4 阶 B 树的个数是（　）。

A.7

B.8

C.9

D.10

【皮皮灰】C.9

4 阶 B 树约束条件

约束项	规则
每个结点关键字数	1～3 个
孩子数	关键字数+1
叶结点特性	都在同一层
根结点特殊性	可只有 1 个关键字

按高度分类讨论

高度为 1（不可能）

- 根结点最多 3 个关键字 < 7 个**排除此情况**

高度为 2 的情况

分析框架

总关键字数 = 根关键字数 + 所有叶结点关键字数 = 7

根关键字数	叶结点数	叶关键字总数	可能的分配方案	方案数
1	2	6	(3,3)	1
2	3	5	(3,1,1)、(1,3,1)、(1,1,3) (2,2,1)、(2,1,2)、(1,2,2)	6
3	4	4	(1,1,1,1)	1

高度为 2 小计：1 + 6 + 1 = 8 种

高度为 3 的情况

最小结构分析

层次分布：
第 1 层（根）：1 个关键字
第 2 层（内部结点）：2 个结点，各 1 个关键字 = 2 个关键字
第 3 层（叶结点）：4 个结点，各 1 个关键字 = 4 个关键字

高度为 3 小计：1 种

高度≥4（不可能）

- 最小 4 层结构需要关键字数：1+2+4+8=15 > 7 **排除此情况**

最终统计

高度	结构数量	备注
2	8	主要情况
3	1	最小结构
总计	**9**	8+1=9

【答案】C.9

1. **分层思考**：根据 B 树高度分类讨论
2. **约束检验**：每层关键字数量必须符合 4 阶 B 树规则
3. **穷举验证**：逐一验证每种可能的关键字分配
4. **边界情况**：注意最小和最大高度的限制

9. 下列关于散列法处理冲突的叙述中，正确的是（　）。

A.只要线性表不满，线性探查再散列一定能找到一个空闲位置

B.只要线性表不满，二次探查再散列一定能找到一个空闲位置

C.线性探查再散列处理的冲突，一定是发生在同义词之间

D.二次探查再散列处理的冲突，一定是发生在非同义词之间

【答案】A.只要线性表不满，线性探查再散列一定能找到一个空闲位置

探查方法对比分析

线性探查再散列

特性项	描述	关键点
探查序列	h(k), h(k)+1, h(k)+2, ..., h(k)+i	步长固定为 1
遍历能力	能遍历整个散列表	☆**核心优势**
空位查找	表未满时必能找到空位	**选项 A 正确**
冲突类型	同义词 + 非同义词	探查过程中都可能遇到

二次探查再散列

特性项	描述	关键点
探查序列	h(k), h(k)±1^2, h(k)±2^2, h(k)±3^2	步长为平方数
遍历能力	**不能保证遍历所有位置**	⚠**关键缺陷**
空位查找	表未满也可能找不到空位	**选项 B 错误**
冲突类型	同义词 + 非同义词	与线性探查相同

选项详细分析

A.只要线性表不满，线性探查再散列一定能找到一个空闲位置✅正确

证明逻辑：
线性探查序列：h(k), h(k)+1, h(k)+2, ..., (h(k)+m-1) mod m
→ 遍历整个表的所有位置
→ 表未满 ⇒ 必存在空位 ⇒ 必能找到

B.只要线性表不满，二次探查再散列一定能找到一个空闲位置❌错误

反例说明：
- 表长 m=8，h(k)=0
- 二次探查序列：0, 1, 4, 1, 4, 1, 4, ...
- 只能访问位置{0,1,4}，无法访问{2,3,5,6,7}
- 即使位置 2 空闲也找不到

C.线性探查再散列处理的冲突，一定是发生在同义词之间❌错误

反例：
1. 关键字 k1: h(k1)=3，存储在位置 3
2. 关键字 k2: h(k2)=0，但位置 0,1,2 被占用
3. 线性探查：0→1→2→3，在位置 3 与 k1 冲突
4. k1 和 k2 的 h 值不同，属于非同义词冲突

D.二次探查再散列处理的冲突，一定是发生在非同义词之间❌错误

反例：
关键字 k1, k2 都有 h(k1)=h(k2)=5（同义词）
k2 进行二次探查时会与 k1 发生冲突

灰灰考研 408 统考 2025 真题

冲突类型说明

冲突类型	定义	在探查中的表现
同义词冲突	h(k1) = h(k2)	初始散列地址相同
非同义词冲突	h(k1) ≠ h(k2)	探查过程中碰撞

重要结论：无论线性探查还是二次探查，都可能处理两种类型的冲突

【答案】A.只要线性表不满，线性探查再散列一定能找到一个空闲位置

关键知识点

1. 遍历完整性
   - **线性探查**：步长为 1，能遍历所有位置
   - **二次探查**：步长为平方数，通常只能遍历部分位置
2. 空位查找保证
   - **线性探查**：表未满 ⇒ 必能找到空位
   - **二次探查**：表未满 ⇏ 必能找到空位
3. 冲突处理范围
   - 两种方法都处理：同义词冲突 + 非同义词冲突

实际应用建议

方法	优点	缺点	适用场景
线性探查	实现简单，必找到空位	容易聚集	负载因子较低时
二次探查	减少聚集现象	可能找不到空位	需要特殊设计保证遍历性

10. 下列排序算法中，最坏情况下元素移动最少的是（　）。

A.起泡排序

B.直接插入排序

C.快速排序

D.简单选择排序

【答案】D.简单选择排序

各排序算法分析

1. 冒泡排序

- **最坏情况**：序列完全逆序
- **移动次数**：3n(n-1)/2
- **原理**：相邻元素两两比较交换，每次交换需要 3 次移动操作

2. 直接插入排序

- **最坏情况**：序列完全逆序
- **移动次数**：n(n-1)/2
- **原理**：每次插入都需要移动已排序部分的所有元素

3. 快速排序

- **最坏情况**：序列已有序或每次基准选择最值
- **移动次数**：$O(n^2)$级别
- **原理**：划分过程中的元素交换，移动次数与比较次数同数量级

4. 简单选择排序

- **最坏情况**：任意初始序列
- **移动次数**：3(n-1)
- **原理**：每次只需交换一次（选中的最值与当前位置），每次交换 3 次移动

移动次数对比表

排序算法	最坏情况移动次数	数量级	备注
冒泡排序	3n(n-1)/2	$O(n^2)$	每次比较可能交换
直接插入排序	n(n-1)/2	$O(n^2)$	插入时移动元素
快速排序	$O(n^2)$	$O(n^2)$	划分时交换操作
简单选择排序	**3(n-1)**	**O(n)**	**仅交换操作**

【答案】D.简单选择排序

简单选择排序在最坏情况下移动次数最少，仅为 3(n-1)次，呈线性关系。这是因为：

1. **固定交换次数**：无论初始序列如何，都只需要 n-1 次交换
2. **每次交换固定移动**：每次交换恰好需要 3 次移动操作
3. **与数据分布无关**：移动次数不受初始序列排列影响

相比之下，其他三种算法在最坏情况下的移动次数都是 $O(n^2)$级别，远大于简单选择排序的 O(n)级别。

常见排序算法比较表

排序算法	平均时间复杂度	最坏时间复杂度	最好时间复杂度	空间复杂度	稳定性	最坏情况移动次数
冒泡排序	$O(n^2)$	$O(n^2)$	O(n)	O(1)	稳定	3n(n-1)/2
直接插入排序	$O(n^2)$	$O(n^2)$	O(n)	O(1)	稳定	n(n-1)/2
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	不稳定	3(n-1)
希尔排序	O(n^1.3)	$O(n^2)$	O(n)	O(1)	不稳定	依赖增量序列
快速排序	O(nlogn)	$O(n^2)$	O(nlogn)	O(logn)	不稳定	$O(n^2)$
归并排序	O(nlogn)	O(nlogn)	O(nlogn)	O(n)	稳定	O(nlogn)
堆排序	O(nlogn)	O(nlogn)	O(nlogn)	O(1)	不稳定	O(nlogn)
基数排序	O(d×n)	O(d×n)	O(d×n)	O(n+k)	稳定	O(n)

排序算法分类

按复杂度分类

- **$O(n^2)$算法**：冒泡、插入、选择排序 - 适合小规模数据
- **O(nlogn)算法**：快速、归并、堆排序 - 适合大规模数据
- **O(n)算法**：基数、桶排序 - 特定条件下使用

按稳定性分类

- **稳定排序**：冒泡、插入、归并、基数排序
- **不稳定排序**：选择、希尔、快速、堆排序

按存储方式分类

- **内部排序**：所有数据在内存中处理
- **外部排序**：数据量大，需要外存辅助

算法选择建议

应用场景	推荐算法	理由
小规模数据(n<50)	直接插入排序	实现简单，常数小
大规模数据	快速排序	平均性能最佳
要求稳定性	归并排序	稳定且时间复杂度优
内存受限 选取 top	堆排序	空间复杂度 O(1)
移动代价大	简单选择排序	移动次数最少
整数排序	基数排序	线性时间复杂度

关键特性总结

- **移动次数最少**：简单选择排序 - 3(n-1)次
- **比较次数最少**：归并排序 - nlogn 次
- **最稳定高效**：归并排序 - 任何情况下 O(nlogn)
- **平均最快**：快速排序 - 实际应用中表现最佳
- **原地排序**：除归并排序外，其他主要算法都是原地排序

11.对含 9 个关键字的初始序列进行排序，若序列的变化情况如下表所示，则下列排序算法中，采用的是（　）。

初始序列	5，25，40，30，10，20，45，15，35
第 1 趟排序后的序列	5，10，20，30，15，35，45，25，40
第 2 趟排序后的序列	5，10，15，25，20，30，40，35，45

A.希尔排序

B.基数排序

C.归并排序

D.折半插入排序

【答案】A.希尔排序

各算法特征对比分析

A. 希尔排序✅正确

核心思想：按增量序列进行分组插入排序，逐步缩小增量

第 1 趟：增量 gap=3

分组情况：

组别	位置索引	初始元素	插入排序后
组 1	0,3,6	5,30,45	5,30,45
组 2	1,4,7	25,10,15	10,15,25
组 3	2,5,8	40,20,35	20,35,40

第 1 趟结果重组：5,10,20,30,15,35,45,25,40

第 2 趟：增量 gap=2

分组情况：

组别	位置索引	第 1 趟后元素	插入排序后
组 1	0,2,4,6,8	5,20,15,45,40	5,15,20,40,45
组 2	1,3,5,7	10,30,35,25	10,25,30,35

第 2 趟结果重组：5,10,15,25,20,30,40,35,45

B. 基数排序❌错误

特征：按位数进行分配和收集，每趟后相同位数的数字会聚集

分析：观察序列变化，不符合按个位数、十位数重新排列的特征

正确的序列【皮皮灰】低位优先，先按个位、再按十位

基数排序按数位进行分配和收集，先按个位排序，再按十位排序。

基数排序过程

阶段	序列	说明
初始序列	5,25,40,30,10,20,45,15,35	
第 1 趟（按个位）	**10,20,30,40,5,25,45,15,35**	个位 0,0,0,0,5,5,5,5,5
第 2 趟（按十位）	**5,10,15,20,25,30,35,40,45**	十位 0,1,1,2,2,3,3,4,4

特征：每趟按特定数位重新分配，最终完全有序

C. 归并排序❌错误

特征：将序列分成若干段，分别排序后合并

分析：第 1 趟后序列不呈现明显的分段有序特征，不符合归并排序模式

正确的序列【皮皮灰】

归并排序采用分治策略，将序列分成小段排序后合并。

归并排序过程（自底向上）

阶段	序列	说明
初始序列	5,25,40,30,10,20,45,15,35	
第 1 趟（两两归并）	**5,25,30,40,10,20,15,45,35**	[5,25],[30,40],[10,20],[15,45],35
第 2 趟（四四归并）	**5,25,30,40,10,15,20,45,35**	[5,25,30,40],[10,15,20,45],35
第 3 趟（最终归并）	**5,10,15,20,25,30,35,40,45**	完全有序

特征：每趟产生更大的有序段，分段有序特征明显

D. 折半插入排序❌错误

特征：逐个插入元素，每趟只处理一个元素

分析：第 1 趟就有多个元素位置发生变化，不符合插入排序特征

正确的序列【皮皮灰】

折半插入排序每次处理一个元素，使用二分查找确定插入位置。

折半插入排序过程

阶段	序列	说明
初始序列	5,25,40,30,10,20,45,15,35	
第 1 趟（插入 25）	**5,25,40,30,10,20,45,15,35**	25 已在正确位置
第 2 趟（插入 40）	**5,25,40,30,10,20,45,15,35**	40 已在正确位置
第 3 趟（插入 30）	**5,25,30,40,10,20,45,15,35**	30 插入到 25 后
第 4 趟（插入 10）	**5,10,25,30,40,20,45,15,35**	10 插入到 5 后
第 5 趟（插入 20）	**5,10,20,25,30,40,45,15,35**	20 插入到 10 后

特征：每趟只处理一个元素，前面部分始终保持有序

【答案】A.希尔排序

排序算法	主要特征	每趟变化规律
希尔排序	按增量分组插入	跨距离移动，渐进有序
基数排序	按数位分配收集	按位数规律重排
归并排序	分治合并	分段有序，段长度倍增
折半插入排序	逐个元素插入	前缀有序，每次+1 个元素

关键识别要点

希尔排序识别特征

- ✓ 多个元素同时大幅移动
- ✓ 不是完全按大小顺序调整
- ✓ 体现分组处理特点

其他算法识别特征

- 基数排序：严格按数位规律排列
- 归并排序：明显的分段有序特征
- 折半插入排序：每趟仅一个元素位置改变

12.在 32 位计算机上执行下列 C 语言代码，则 ui 的真值为（　）。

```
short si=-32767;
unsigned int ui=si;
```

A.$2^{15}-1$

B.$2^{15}+1$

C.$2^{32}-2^{15}-1$

D.$2^{32}-2^{15}+1$

【答案】D. $2^{32}-2^{15}+1$

数据类型规格

类型	位数	取值范围	表示方法
short	16 位	-32768 ~ 32767	二进制补码
unsigned int	32 位	0 ~ $2^{32}-1$	无符号二进制

转换过程分析

第一步：short 类型的二进制表示

- **原值**：si = -32767
- **32767 的二进制**：0111 1111 1111 1111
- **-32767 的补码**：1000 0000 0000 0001

第二步：符号扩展到 32 位

将 16 位的 short 扩展到 32 位时，符号位扩展：

```
16 位: 1000 0000 0000 0001
32 位: 1111 1111 1111 1111 1000 0000 0000 0001
```

第三步：无符号解释

32 位二进制 **1111 1111 1111 1111 1000 0000 0000 0001** 按无符号整数解释：

计算方法	表达式	结果
补码转换	2^{32} + (-32767)	2^{32} - 32767
数学变换	2^{32} - (2^{15} - 1)	2^{32} - 2^{15} + 1

【答案】D. $2^{32}-2^{15}+1$

验证计算

方法一：直接计算

$$
\begin{aligned}
ui &= 2^{32} - 32767 \\
&= 2^{32} - (2^{15} - 1) \\
&= 2^{32} - 2^{15} + 1
\end{aligned}
$$

方法二：位模式验证

```
32 位补码: 11111111111111111000000000000001
转无符号: 4294934529 = 2^32 - 2^15 + 1
```

原理：负数的 short 类型转换为 unsigned int 时，按照模 2^{32}运算，结果为 2^{32} + 原值 = 2^{32} + (-32767) = 2^{32} - 2^{15} + 1。

13. 已知 float 型变量用 IEEE754 单精度浮点数格式表示。

若 float 型变量 x 的机器数为 4730 0000H；则 x 的值为（ ）。

A. . .375×2^{14}

B. . .375×2^{14}

C. . .375×2^{15}

D. .375×2^{15}

【答案】D. 1.375×2^{15}

IEEE754 单精度格式

组成部分	位数	位置	作用
符号位 S	1 位	第 31 位	0 为正，1 为负
指数位 E	8 位	第 30-23 位	偏移 127 的指数
尾数位 M	23 位	第 22-0 位	小数部分

数值计算公式： $(-1)^S \times (1 + M) \times 2^{(E-127)}$

机器数解析

十六进制转二进制

4730 0000H = 0100 0111 0011 0000 0000 0000 0000 0000B

各字段提取

字段	二进制值	十进制值	说明
符号位 S	0	0	正数
指数位 E	10001110	142	8 位指数
尾数位 M	01100000000000000000000	-	23 位尾数

数值计算

指数计算

$E = 10001110_2 = 1\times2^7 + 0\times2^6 + 0\times2^5 + 0\times2^4 + 1\times2^3 + 1\times2^2 + 1\times2^1 + 0\times2^0$

$= 128 + 8 + 4 + 2 = 142$

真实指数 $= E - 127 = 142 - 127 = 15$

尾数计算

$M = 01100000000000000000000_2$

$= 0\times2^{-1} + 1\times2^{-2} + 1\times2^{-3} + 0\times2^{-4} + \ldots$

$= 0 + 0.25 + 0.125 + 0$

$= 0.375$

完整尾数 $= 1 + M = 1 + 0.375 = 1.375$

最终结果

$x = (-1)^0 \times 1.375 \times 2^{15} = 1.375 \times 2^{15}$

【答案】D. 1.375×2^{15}

关键点：

- IEEE754 格式中尾数隐含前导 1，实际尾数为 1.375
- 指数部分需减去偏置值 127，得到真实指数 15
- 符号位为 0 表示正数

14. 假设 8 位字长的计算机中，两个带符号整数 x 和 y 的补码表示分别为$[x]_{补}$=A3H，$[y]_{补}$=75H，则通过补码加减运算器得到的 x-y 的值及 OF 标志分别为（ ）。

A. 24，0

B. 24，1

C. 46，0

D. 46，1

【答案】D. 46，1

1. 确定 x 和 y 的真值

- **[x]补 = A3H = 10100011_2**
  - MSB = 1，为负数
  - 真值：x = A3H - 256 = 163 - 256 = **-93**
- **[y]补 = 75H = 01110101_2**
  - MSB = 0，为正数
  - 真值：y = 75H = **+117**

2. 理论计算 x-y

x - y = -93 - 117 = **-210**

由于 8 位补码范围为[-128, 127]，-210 超出范围，**发生溢出**。

3. 补码加减运算器实现

补码减法通过加法实现：**x - y = x + (-y)**

求-y 的补码：

- y = 75H = 01110101_2
- 按位取反：10001010_2
- 加 1：10001011_2 = **8BH**

执行加法运算：

```
  A3H          10100011
+ 8BH        + 10001011
————————     ————————————

 12EH         100101110
```

截断为 8 位：**2EH = 46**

4. 溢出标志判断

项目	符号位	说明
x	1	负数
-y	1	负数
结果	0	正数

两个负数相加得到正数 → OF = 1（溢出）

【答案】D. 46，1

15. 某 32 计算机按字节编制，采用小端方式存放数据，编译器按边界对齐方式为下列 c 语言结构型数组变量 emplogee 分配储存空间。

```
struct record {
        int id;
        char name[10];
        int salary;
} employee[200];
```

数组 employee 的起始地址为 0000 A0B0H，employee[1].id 的机器数为 12345678H，则 56H 的地址是（ ）。

A. . 000 A0C3H

B. . 000 A0C4H

C. . 000 A0C5H

D. 000 A0C6H

【答案】C. 0000 A0C5H

1. 结构体内存对齐分析

在 32 位系统中，需要按边界对齐方式分配存储空间：

成员	数据类型	大小	偏移地址	对齐要求
id	int	4 字节	0	4 字节对齐
name[10]	char[10]	10 字节	4	1 字节对齐
填充	-	2 字节	14	为 salary 对齐
salary	int	4 字节	16	4 字节对齐

结构体总大小：4 + 10 + 2 + 4 = **20 字节**

2. 数组元素地址计算

- **employee[0]** 起始地址：**0000 A0B0H**
- **employee[1]** 起始地址：**A0B0H + 20 = 0000 A0C4H**
- **employee[1].id** 地址：**0000 A0C4H**

3. 小端存储方式分析

employee[1].id 的机器数为 **12345678H**，采用小端方式存储：

地址	存储值	说明
A0C4H	78H	最低字节
A0C5H	**56H**	次低字节
A0C6H	34H	次高字节
A0C7H	12H	最高字节

4. 关键知识点

内存对齐规则

- **基本类型对齐：**int 型按 4 字节边界对齐
- **结构体对齐：**成员按各自对齐要求排列，可能需要填充字节
- **数组对齐：**结构体大小必须是最大成员对齐值的倍数

小端存储特点

- **字节顺序：**低位字节存储在低地址
- **多字节数据：**按字节倒序存储
- **地址递增：**从最低有效字节到最高有效字节

【答案】C. 0000 A0C5H

字节 **56H** 作为 **12345678H** 的次低字节，存储在地址 **0000 A0C5H** 处。

16. 下列选项中，由指令集体系结构（ISA）规定的是（ ）。

A.是否采用阵列乘法器

B.是否采用定长指令字格式

C.是否采用微程序控制器

D.是否采用单总线数据通路

【答案】B.是否采用定长指令字格式

ISA 与微架构的区别

层次	ISA（指令集体系结构）	微架构（实现层）
定义	软件可见的体系结构接口	硬件具体实现方式
特点	抽象、标准化、向后兼容	具体、可变、性能导向
影响	决定软件兼容性	决定硬件性能

各选项分析

A. 是否采用阵列乘法器 ❌**错误**

- **归属**：微架构实现
- **说明**：乘法器类型（阵列、移位加、布斯算法等）是硬件实现细节
- **ISA 视角**：只关心乘法指令功能，不关心具体实现方式

B. 是否采用定长指令字格式 ✅正确

- **归属**：ISA 规范
- **说明**：指令编码长度直接影响指令解析和程序兼容性
- **重要性**：软件必须知道指令长度才能正确解析程序

C. 是否采用微程序控制器 ❌**错误**

- **归属**：微架构实现
- **说明**：控制器设计（微程序 vs 硬布线）是内部实现选择
- **ISA 视角**：只要能正确执行指令即可，不关心控制方式

D. 是否采用单总线数据通路 ❌**错误**

- **归属**：微架构实现
- **说明**：数据通路设计是硬件性能优化手段
- **ISA 视角**：只关心数据传输结果，不关心通路结构

ISA 核心规定内容

软件可见特性

- **指令格式**：编码方式、长度规范
- **指令类型**：运算、访存、控制转移等
- **寻址方式**：立即数、寄存器、间接等
- **数据类型**：整数、浮点数、字符等

系统状态

- **寄存器集合**：通用寄存器、状态寄存器
- **存储模型**：地址空间、字节序
- **异常处理**：中断、异常的软件接口

【答案】B.是否采用定长指令字格式

指令字格式属于 ISA 必须规定的软件可见特性，直接影响指令解析和程序兼容性。而其他选项均为微架构实现细节，不由 ISA 规定。

17. 下列关于 RISC 的叙述中，错误的是（ ）。

A.多采用硬连线方式实现控制器

B.通常采用 Load/Store 型指令设计风格

C.难以采用流水线数据通路实现微架构

D.多采用寄存器传递过程调用时的参数

【答案】C.难以采用流水线数据通路实现微架构

1. RISC 设计原则与特点

特性	RISC 设计理念	优势
指令复杂度	简单、基本指令	易于硬件实现
指令长度	固定长度	便于流水线设计
指令格式	统一格式	简化译码逻辑
寻址方式	简单寻址	减少执行周期

2. 各选项详细分析

A. 多采用硬连线方式实现控制器 ✅正确

- **原因**：RISC 指令简单，控制逻辑相对固定
- **优势**：执行速度快，硬件开销小
- **对比**：CISC 多用微程序控制，灵活但速度慢

B. 通常采用 Load/Store 型指令设计风格 ✅正确

- **定义**：只有 Load/Store 指令访问内存
- **特点**：算术逻辑运算只在寄存器间进行
- **好处**：简化指令设计，提高执行效率

C. 难以采用流水线数据通路实现微架构 ❌错误

- **实际情况**：RISC **最适合**流水线实现
- **设计目标**：RISC 正是为高效流水线而生
- **支撑因素**：
  - 指令长度固定，便于取指
  - 指令格式统一，简化译码
  - 操作简单，执行周期可预测
  - 寻址方式简单，减少地址计算复杂度

D. 多采用寄存器传递过程调用时的参数 ✅正确

- **策略**：参数通过寄存器传递
- **优势**：避免频繁的栈操作，提高效率
- **实现**：RISC 通常提供较多通用寄存器

3. RISC 与流水线的完美结合

流水线友好特性

RISC 特性	对流水线的益处
固定指令长度	取指阶段时间固定
统一指令格式	译码逻辑简化
简单操作	执行时间可预测
Load/Store 结构	访存操作规范化

流水线冲突处理

- **数据冲突**：通过寄存器重命名和前递技术解决
- **控制冲突**：分支预测和延迟槽技术
- **结构冲突**：资源复制和调度优化

4. RISC vs CISC 对比

特性	RISC	CISC
指令复杂度	简单、基本	复杂、功能丰富
控制器	硬连线为主	微程序为主
流水线	易于实现	实现复杂
寄存器数量	多（32+）	少（8-16）

5. 关键知识点

RISC 设计哲学

- **简单就是美**：通过软件复杂性换取硬件简单性
- **常用指令优化**：80-20 原则，优化常用操作
- **编译器友好**：为编译器优化提供良好支持

现代发展趋势

- **RISC-V**：开源指令集架构
- **ARM**：移动设备主流架构
- **超标量**：多发射流水线技术

【答案】C.难以采用流水线数据通路实现微架构

RISC 设计的核心目标之一就是便于流水线实现，其简单统一的指令特性使其**最适合**流水线架构，而非"难以实现"。

18.下列关于 CPI 和 CPU 时钟周期的叙述中，错误的是（ ）。
A.不同类型指令的 CPI 可能不一样
B.程序的 CPI Cache 缺失率无关
C.单周期 CPU 的时钟周期以最耗时指令所用的时间为准
D.流水线 CPU 的时钟周期以最长流水段所用时间为准
【答案】B.程序的 CPI Cache 缺失率无关

1. CPI 基本概念

CPI (Cycles Per Instruction)：每条指令的平均时钟周期数

CPU 执行时间公式：

CPU 时间 = 指令数 × CPI × 时钟周期

2. 各选项详细分析

A. 不同类型指令的 CPI 可能不一样 ✅正确

指令类型	典型 CPI	原因
简单运算	1-2	ALU 操作简单
乘除法	3-10	复杂运算需要多周期
访存指令	1-100+	取决于存储层次
分支指令	1-5	可能涉及分支预测失败

B. 程序的 CPI 与 Cache 缺失率无关 ❌错误

实际情况：CPI 与 Cache 缺失率**高度相关**

Cache 状态	影响	CPI 变化
Cache 命中	1-2 周期访存	基准 CPI
L1 缺失	~10 周期等待	CPI 增加
L2 缺失	~100 周期等待	CPI 大幅增加
主存访问	200+周期	CPI 显著恶化

C. 单周期 CPU 的时钟周期以最耗时指令所用的时间为准 ✅正确

- **设计原则**：所有指令在一个时钟周期内完成
- **时钟周期**：必须满足最慢指令的执行要求
- **缺点**：简单指令被拖慢，整体性能不佳

D. 流水线 CPU 的时钟周期以最长流水段所用时间为准 ✅正确

- **流水线约束**：所有段同步工作
- **时钟周期**：由最慢的流水段决定
- **优化目标**：均衡各段执行时间

3. Cache 对 CPI 的具体影响

缺失惩罚计算

实际 CPI = 基准 CPI + 缺失率 × 缺失惩罚

多级缓存影响

缓存级别	命中率	缺失惩罚	CPI 贡献
L1 I-Cache	~95%	10 周期	+0.5
L1 D-Cache	~90%	10 周期	+1.0
L2 Cache	~99%	100 周期	+1.0
主存	~99.9%	200 周期	+0.2

4. 不同 CPU 架构的时钟周期设计

单周期 CPU

- **时钟周期** = MAX(取指，译码，执行，访存，写回)
- **问题**：资源利用率低，性能受限

流水线 CPU

- **时钟周期** = MAX(各流水段时间)
- **优势**：提高吞吐量
- **挑战**：需要处理冒险和冲突

超标量 CPU

- **时钟周期**：由最复杂执行单元决定
- **特点**：多指令并发执行

5. 影响 CPI 的主要因素

因素类别	具体因素	影响程度
指令特性	指令类型分布	高
存储系统	Cache 缺失率	**极高**
流水线	数据冒险、控制冒险	中等
分支预测	预测准确率	中等

6. 关键知识点

CPI 优化策略

- **指令级**：减少复杂指令使用
- **编译器级**：优化指令调度和寄存器分配
- **硬件级**：改进 Cache 设计，提高命中率
- **系统级**：优化数据局部性

性能评估

- **理论 CPI**：不考虑存储系统延迟
- **实际 CPI**：包含各种停顿和等待
- **平均 CPI**：加权平均各类指令的 CPI

【答案】B.程序的 CPI Cache 缺失率无关

Cache 缺失会导致流水线停顿和长时间等待，**显著影响**程序的实际 CPI。缺失率越高，CPI 越大，这是现代处理器性能分析的关键因素。

19. 下列关于 CPU 中的数据通路和控制器的叙述中，错误的是（ ）。
A.通用寄存器组中应该包含程序计数器
B.控制器中一定包含指令操作码的译码电路
C.单周期 CPU 中的控制器比多周期 CPU 中的更简单
D.流水线 CPU 需解决数据相关和控制相关等冒险问题
【答案】A.通用寄存器组中应该包含程序计数器

1. CPU 核心组件分类

组件类别	具体组件	功能特点
通用寄存器	R0-R31 等	存储操作数，程序可直接访问
专用寄存器	PC、SP、PSW 等	特殊功能，系统管理
控制器	译码器、控制逻辑	指令解析与控制信号生成
数据通路	ALU、总线、多路器	数据传输与运算

2. 各选项详细分析

A. 通用寄存器组中应该包含程序计数器 ❌错误

PC 的特殊性质：

- **独立性**：PC 是独立的专用寄存器
- **自动更新**：硬件自动管理，不需程序直接操作
- **访问方式**：通过跳转指令间接访问，非通用读写

寄存器分类对比：

寄存器类型	例子	访问方式	用途
通用寄存器	R1, R2, R3	直接寻址	存储操作数
专用寄存器	PC, SP, PSW	特殊指令	系统控制

B. 控制器中一定包含指令操作码的译码电路 ✅正确

译码器的必要性：

- **功能**：将操作码转换为控制信号
- **实现方式**：
  - 硬布线控制：组合逻辑电路
  - 微程序控制：微指令存储器
- **输出**：ALU 控制、寄存器使能、数据通路选择等信号

C. 单周期 CPU 中的控制器比多周期 CPU 中的更简单 ✅正确

控制复杂度对比：

CPU 类型	控制特点	复杂度原因
单周期	纯组合逻辑	无状态控制，直接译码
多周期	有限状态机	需状态转换，时序控制
流水线	多级控制	冒险检测，流水线控制

D. 流水线 CPU 需解决数据相关和控制相关等冒险问题 ✅正确

流水线冒险类型：

冒险类型	产生原因	解决方案
数据冒险	指令间数据依赖	前递、停顿、重排序
控制冒险	分支跳转	分支预测、延迟槽
结构冒险	资源冲突	资源复制、调度

3. 程序计数器(PC)详解

PC 的特殊地位

- **硬件管理**：取指时自动递增
- **间接访问**：通过跳转指令修改
- **系统级别**：属于处理器状态的一部分

与通用寄存器的区别

```
通用寄存器：ADD R1, R2, R3      // 直接指定寄存器
程序计数器：JMP LABEL           // 间接修改 PC 值
```

4. 不同 CPU 架构的控制器设计

单周期 CPU 控制器

- 设计：纯组合逻辑
- 输入：指令操作码
- 输出：所有控制信号
- 特点：简单但功耗高

多周期 CPU 控制器

- 设计：有限状态机
- 状态：取指→译码→执行→访存→写回
- 复杂性：需要状态转换逻辑

流水线 CPU 控制器

- 设计：分布式控制
- 特点：每级流水段有独立控制
- 挑战：冒险检测与处理

5. 数据通路设计要点

关键组件

- ALU：算术逻辑运算
- 寄存器文件：数据存储
- 多路器：数据路径选择
- 总线：数据传输通道

设计原则

- 数据流向：明确输入输出路径
- 控制信号：确保正确的数据传输
- 时序约束：满足建立和保持时间

6. 关键知识点

寄存器文件设计

- 读端口：支持并发读取多个操作数
- 写端口：结果写回控制
- 旁路：解决读写冲突

控制器设计方法

- 硬布线：速度快，但灵活性差
- 微程序：灵活，但速度相对慢
- 混合方式：结合两种优势

【答案】A.通用寄存器组中应该包含程序计数器

程序计数器(PC)是专用寄存器，由硬件自动管理，不属于程序可直接访问的通用寄存器组。通用寄存器主要用于存储操作数，而 PC 用于指令地址管理。

20. 某处理器总线采用同步，并行传输方式，每个总线时钟周期传送 4 次数据（quadpumped技术），若该总线的工作频率为 1333MHz（实际单位是 MT/s，表示每秒传送 1333M/次），总线宽度为 64 位，则总线带宽约为（ ）。

A.10.66GB/s　B.42.66GB/s　C.85.31GB/s　D.341.25 GB/s

【答案】A.10.66GB/s

1. 题目关键信息分析

参数	数值	说明
传输方式	同步并行	时钟同步，多位并行
工作频率	1333 MT/s	每秒传输 1333M 次
总线宽度	64 位	每次传输 8 字节
技术	Quad-pumped	每时钟周期 4 次传输

2. 带宽计算步骤

基本公式

总线带宽 = 传输频率 × 数据宽度

具体计算

- **有效传输频率**：1333 MT/s = 1333×10^6 次/秒
- **每次传输数据量**：64 位 = 8 字节
- **总线带宽**：$1333 \times 10^6 \times 8 = 10{,}664 \times 10^6$ B/s

单位换算

$10{,}664 \times 10^6$ B/s = 10,664 MB/s ≈ 10.66 GB/s

3. 关键技术解析

Quad-pumped 技术

- **定义**：每个时钟周期传输 4 次数据
- **实现**：利用时钟上升沿、下降沿等多个边沿
- **影响**：题目中 1333 MT/s 已包含此技术效果

MT/s 单位说明

- **含义**：Mega Transfers per Second（每秒百万次传输）
- **特点**：已考虑多倍数据传输技术
- **注意**：与 Hz 频率概念不同

4. 常见总线技术对比

技术类型	传输倍数	频率表示
SDR	1×时钟频率	Hz
DDR	2×时钟频率	MT/s
QDR	4×时钟频率	MT/s

5. 计算易错点

❌ **错误理解：再次乘以 4**

错误：1333 × 4 × 8 = 42.66 GB/s
原因：重复计算 quad-pumped 效果

✅ **正确理解：直接使用有效传输率**

正确：1333 × 8 = 10.66 GB/s
原因：MT/s 已包含传输倍数

【答案】A.10.66GB/s

注意：题目中的 MT/s 单位已包含 quad-pumped 技术的传输倍增效果，无需额外乘以 4。

21. 适合采用 DMA 的设备有（ ）。

I.键盘　II.网卡　III.固态硬盘　IV.针式打印机

A.I、II　　B.II、III　　C.II、IV　　D.III、IV

【答案】B.II、III

1. DMA 技术特点

DMA (Direct Memory Access)：设备直接访问内存，无需 CPU 干预

DMA 优势与开销

方面	特点	影响
优势	CPU 解放，高速传输	提高系统效率
开销	初始化复杂，硬件成本	需要权衡投入产出
适用	大块数据，高频传输	成本效益明显

2. I/O 设备分类与传输特性

设备类型	数据量	传输频率	速度需求	典型设备
块设备	大（KB-MB）	中等	高	硬盘、SSD
网络设备	大（KB-MB）	高	极高	网卡、光纤卡
字符设备	小（字节）	低	低	键盘、串口
打印设备	小-中等	低	低-中	各类打印机

3. 各设备 DMA 适用性分析

I. 键盘 ❌错误

- **数据特点**：单字符，1-2 字节
- **传输频率**：极低（人工输入）
- **DMA 问题**：
  - 初始化开销 > 传输收益
  - 实时性要求高，中断更适合
- **最佳方案**：中断驱动

II. 网卡 ✅正确

- **数据特点**：网络包，64B-1500B+
- **传输频率**：极高（Gbps 级别）
- **DMA 优势**：
  - 大量数据块传输
  - 减少 CPU 中断负担
  - 提高网络吞吐量
- **应用**：现代网卡标配 DMA

III. 固态硬盘(SSD) ✅正确

- **数据特点**：数据块，4KB-1MB+
- **传输频率**：高（读写操作频繁）
- **DMA 优势**：
  - 大块数据传输效率高
  - CPU 可并行处理其他任务
  - 充分发挥 SSD 高速特性
- **应用**：SATA、NVMe 协议均支持 DMA

IV. 针式打印机 ❌错误

- **数据特点**：字符流，字节级
- **传输频率**：低（机械打印速度限制）

- **DMA 问题：**
  - 打印速度慢，DMA 优势不明显
  - 字符处理适合程序控制
  - 成本效益不高
- **最佳方案：**程序查询或中断

4. DMA 适用性判断标准

适合 DMA 的条件

判断标准	阈值/特征	说明
数据块大小	>1KB	摊销 DMA 初始化成本
传输频率	高频或突发	充分利用 DMA 效率
CPU 占用	传输时间长	释放 CPU 处理其他任务
实时性	中等容忍度	DMA 有启动延迟

不适合 DMA 的情况

- **小数据量：**字符级传输
- **低频传输：**偶发性数据交换
- **强实时性：**需要立即响应
- **简单设备：**成本敏感应用

5. 现代 I/O 发展趋势

高速设备 DMA 应用

- **NVMe SSD：**多队列 DMA
- **高速网卡：**RDMA 技术
- **GPU：**显存 DMA 传输
- **存储阵列：**RAID 控制器 DMA

DMA 优化技术

- **Scatter-Gather：**分散聚集传输
- **Chain DMA：**链式 DMA 传输
- **IOMMU：**I/O 内存管理单元

6. 关键知识点

DMA 工作流程

1. **初始化：**设置传输参数
2. **启动：**设备请求总线
3. **传输：**直接内存访问
4. **完成：**中断通知 CPU

DMA vs 其他 I/O 方式

方式	CPU 占用	传输效率	适用场景
程序 I/O	100%	低	简单控制
中断 I/O	中等	中等	小数据传输
DMA	最小	高	大块数据传输

【答案】B.II、III

网卡和固态硬盘都属于高速、大数据量传输设备，DMA 技术能显著提高传输效率并减少 CPU 负担。而**键盘和针式打印机**属于低速字符设备，使用 DMA 的成本效益不高。

22. 下列选项中，会触发外部中断请求的事件是（ ）。

A.DMA 传送结束

B.总线事务结束

C.页故障的处理结束

D.执行断点指令

【答案】A.DMA 传送结束

1. 中断分类体系

中断类型	触发源	特点	典型事件
外部中断	外设/硬件	异步，不可预测	I/O 完成，定时器
内部中断	CPU 内部	同步，指令相关	异常，陷入
软件中断	程序指令	主动触发	系统调用

2. 各选项详细分析

A. DMA 传送结束 ✅ 外部中断

- **触发机制**：DMA 控制器完成数据传输后向 CPU 发送中断信号
- **中断源**：外部硬件设备（DMA 控制器）
- **目的**：通知 CPU 传输完成，可进行后续处理
- **时机**：异步发生，CPU 无法预知确切时间

B. 总线事务结束 ❌ 非中断事件

- **性质**：总线协议层面的握手信号
- **处理方式**：硬件自动处理，无需 CPU 介入
- **特点**：底层传输控制，不产生中断请求

C. 页故障的处理结束 ❌ 内部处理

- **原始事件**：页故障本身是内部异常
- **处理结束**：异常处理程序正常返回
- **机制**：通过异常返回指令恢复，非外部中断

D. 执行断点指令 ❌ 软件中断/陷入

- **触发方式**：CPU 执行特定指令（如 INT 3）
- **中断源**：程序内部指令
- **用途**：调试、系统调用等

3. 外部中断特征

识别要点

- **异步性**：与 CPU 执行流程无关
- **外部源**：来自 CPU 外部设备
- **硬件信号**：通过中断控制器传递
- **不可预测**：发生时机由外设决定

常见外部中断源

设备类型	中断事件	用途
I/O 设备	数据准备就绪	通知 CPU 处理
DMA 控制器	传输完成	释放总线控制权
定时器	时间到达	任务调度
网络设备	数据包到达	网络处理

4. 中断处理流程

DMA 完成中断示例

1. D　A 控制器完成数据传输
2. 向中断控制器发送中断请求
3. 中断控制器通知 CPU
4. C　U 保存现场，跳转中断处理程序
5. 处理 DMA 完成事务
6. 恢复现场，继续原程序执行

【答案】A.DMA 传送结束

DMA 传送结束时，**DMA 控制器**作为外部硬件向 CPU 发送中断请求，属于典型的**外部中断**事件。其他选项要么是内部处理机制，要么不产生中断请求。

23. 在采用页式虚拟存储管理方式的系统中，当发生上下文切换时，下列寄存器中操作系统不需要更新的是（ ）。

A.通用寄存器 B.页表基址寄存器 C.程序计数器 D.中断向量表的基址寄存器

【答案】D.中断向量表的基址寄存器

1. 上下文切换概念

定义与时机

上下文切换是指 CPU 从执行一个进程转换到执行另一个进程时，保存当前进程状态并恢复目标进程状态的过程。

触发时机	说明
时间片用完	分时系统中进程轮转
进程阻塞	I/O 等待、信号量等待等
高优先级进程就绪	抢占式调度
系统调用	主动让出 CPU

2. 寄存器分类与切换需求

需要更新的寄存器

寄存器类型	更新原因	切换时操作
A. 通用寄存器	进程计算状态	保存→加载新值
B. 页表基址寄存器	地址空间切换	指向新进程页表
C. 程序计数器(PC)	执行位置	保存→加载新 PC

A. 通用寄存器 ✅ 需要更新

保存内容：进程的计算中间结果、变量值
切换操作：
1. 保存当前进程的 R0-Rn 到 PCB
2. 从新进程 PCB 加载 R0-Rn

B. 页表基址寄存器 ✅ 需要更新

保存内容：当前进程页表的物理地址
切换操作：
1. 保存当前页表基址
2. 加载新进程页表基址
3. 刷新 TLB(页表缓存)

C. 程序计算器 ✅ 需要更新

保存内容：下一条要执行的指令地址
切换操作：
1. 保存当前 PC 值到 PCB
2. 从新进程 PCB 加载 PC 值

不需要更新的寄存器

D. 中断向量表基址寄存器 ❌ 不需要更新

特征	说明
系统级配置	属于操作系统全局资源
所有进程共享	中断处理程序对所有进程相同
内核空间资源	不属于用户进程上下文

3. 中断向量表特性分析

全局性质

中断向量表 = 系统中所有中断处理程序的入口地址表
特点：
- 由操作系统内核维护
- 所有进程共享同一套中断处理机制
- 位于内核地址空间

典型架构示例

架构	寄存器名	用途
x86	IDTR	中断描述符表基址
ARM	VBAR	向量基址寄存器
MIPS	固定地址	异常向量固定位置

4. 上下文切换完整流程

进程状态保存与恢复

当前进程 → PCB 保存：

```
┌──────────────────────────────────┐
│ 通用寄存器                       │ ← 需要保存
│ 程序计数器                       │ ← 需要保存
│ 状态寄存器                       │ ← 需要保存
│ 页表基址                         │ ← 需要保存
└──────────────────────────────────┘
```

系统级寄存器：

```
┌──────────────────────────────────┐
│ 中断向量表基址                   │ ← 保持不变
│ GDT 基址(x86)                    │ ← 保持不变
│ 控制寄存器                       │ ← 部分不变
└──────────────────────────────────┘
```

5. 内存管理相关寄存器

页式存储管理中的关键寄存器

寄存器	功能	切换时处理
页表基址寄存器(PTBR)	指向当前页表	必须更新
页表长度寄存器(PTLR)	页表大小	需要更新
TLB	页表缓存	需要刷新

地址转换流程

```
虚拟地址 → [页表基址寄存器] → 页表 → 物理地址
              ↑
        进程切换时必须更新
```

【答案】D.中断向量表的基址寄存器

其他选项(A、B、C)都是进程特定的状态信息，在上下文切换时必须保存和恢复，以确保进程能够正确继续执行。

24. 关于虚拟化技术，错误的说法是（ ）。
A.操作系统可以在虚拟机上运行
B.一台主机可以支持多个虚拟机
C.VMM 与操作系统特权级相同
D.通过虚拟机技术，可以用一台主机上模拟多种 ISA
【答案】C. VMM 与操作系统特权级相同

详细解析

虚拟化架构特权级关系

组件	特权级	作用
VMM (虚拟机监控器)	**最高特权级**	控制和管理所有虚拟机资源
Host OS (宿主操作系统)	高特权级	管理物理硬件资源
Guest OS (客户操作系统)	**降权运行**	在虚拟环境中运行
应用程序	用户级	在虚拟机内正常运行

各选项分析

A. 操作系统可以在虚拟机上运行 ✅ 正确
- 正确。Guest OS 可以完整运行在虚拟机环境中
- 通过 VMM 提供的虚拟硬件接口实现

B. 一台主机可以支持多个虚拟机 ✅ 正确
- 正确。这是虚拟化技术的核心优势
- VMM 负责资源分配和隔离管理

C. VMM 与操作系统特权级相同 ❌ 错误
- **错误**。VMM 必须具有比 Guest OS 更高的特权级
- 原因：
  - 需要拦截和处理 Guest OS 的特权指令
  - 控制敏感资源访问
  - 实现虚拟机间的安全隔离

D. 通过虚拟机技术，可以用一台主机上模拟多种 ISA ✅ 正确
- 正确。通过全系统仿真器（如 QEMU）实现
- 属于广义虚拟化技术范畴

【答案】C. VMM 与操作系统特权级相同

特权级分离的必要性：
- VMM 需要完全控制物理资源
- Guest OS 必须在受控环境中运行
- 通过硬件虚拟化扩展或软件陷入机制实现特权指令拦截

25. 某进程采用优先权调度，采用单链表保存进程就绪队列，高优先级进程在队头。就绪队列长度为 n，则插入进程、选出进程的时间复杂度分别是（ ）。

A. O（1），O（1）

B. O（1）、O（n）

C. O（n），O（1）

D. O（n），O（n）

【答案】C. O（n），O（1）

数据结构分析

就绪队列结构

- **存储方式**：单链表
- **排序规则**：按优先级从高到低有序排列
- **队列长度**：n 个进程

```
队头 → [高优先级] → [中优先级] → [低优先级] → NULL
       (最先调度)                    (最后调度)
```

操作复杂度分析

操作	时间复杂度	分析过程
插入进程	**O(n)**	需要遍历链表找到合适位置
选出进程	**O(1)**	直接取队头元素

插入进程操作 - O(n)

操作步骤：

1. 从队头开始遍历链表
2. 比较新进程与当前进程的优先级
3. 找到第一个优先级低于新进程的位置
4. 在该位置前插入新进程

最坏情况：新进程优先级最低，需遍历整个链表

- **时间复杂度：O(n)**

选出进程操作 - O(1)

操作步骤：

1. 直接访问队头指针
2. 取出队头进程（最高优先级）
3. 更新队头指针指向下一个节点

访问特点：无需遍历，直接定位

- **时间复杂度：O(1)**

【答案】C. O（n），O（1）

算法优化思考

当前方案：有序单链表

- 优点：调度快速 O(1)
- 缺点：插入较慢 O(n)

可选优化：

- **堆结构**：插入 O(log n)，选出 O(log n)
- **多级队列**：按优先级分组，减少遍历范围

26. 现有一 LRU 算法，固定分配局部置换，已为进程分配 3 个页框，页面访问序列为{0，1，2，0，5，1，4，3，0，2，3，2，0}，其中 0，1，2 已调入内存。则缺页次数是（ ）。

A.5

B.6

C.7

D.8

【答案】B. 6

详细执行过程

步骤	访问页面	访问前内存状态	LRU 替换顺序	操作结果	访问后内存状态
初始	-	{0,1,2}	-	-	{0,1,2}
1	0	{0,1,2}	-	命中	{0,1,2}
2	1	{0,1,2}	-	命中	{0,1,2}
3	2	{0,1,2}	-	命中	{0,1,2}
4	0	{0,1,2}	1→2→0	命中	{0,1,2}
5	**5**	{0,1,2}	2→0→5	**缺页，替换 1**	{0,5,2}
6	**1**	{0,5,2}	0→5→1	**缺页，替换 2**	{0,5,1}
7	**4**	{0,5,1}	5→1→4	**缺页，替换 0**	{4,5,1}
8	**3**	{4,5,1}	1→4→3	**缺页，替换 5**	{4,3,1}
9	**0**	{4,3,1}	4→3→0	**缺页，替换 1**	{4,3,0}
10	**2**	{4,3,0}	3→0→2	**缺页，替换 4**	{2,3,0}
11	3	{2,3,0}	2→0→3	命中	{2,3,0}
12	2	{2,3,0}	3→0→2	命中	{2,3,0}
13	0	{2,3,0}	2→3→0	命中	{2,3,0}

关键替换分析

缺页发生位置详解

缺页序号	访问页面	被替换页面	替换原因
1	5	1	1 是最近最少使用的页面
2	1	2	2 是最近最少使用的页面
3	4	0	0 是最近最少使用的页面
4	3	5	5 是最近最少使用的页面
5	0	1	1 是最近最少使用的页面
6	2	4	4 是最近最少使用的页面

LRU 算法核心原理

替换策略：每次选择最长时间未被访问的页面进行替换 **实现方式：**维护页面访问的时间序列，最左侧为最久未使用

统计结果

- **总访问次数：13 次**
- **缺页次数：6 次**
- **命中次数：7 次**
- **缺页率：6/13 ≈ 46.15%**

【答案】B. 6

27. 确定进程运行所需的最少页框数时，要考虑的指标是（ ）。

A.代码段长

B.虚拟地址空间大小

C.物理地址空间大小

D.指令系统支持的寻址方式

【答案】D.指令系统支持的寻址方式

指令系统支持的寻址方式

最少页框数：进程能够正常执行所需的最小内存页框数量，确保任何一条指令都能完整执行而不发生无限缺页中断。

关键分析

一条指令执行可能涉及的页面

页面类型	用途	寻址方式影响
指令页	存放当前执行指令	必需，所有指令都要访问
操作数页	存放源操作数	取决于操作数寻址方式
目标页	存放目标操作数	取决于目标寻址方式
间接寻址页	多级间接寻址	取决于间接寻址层数

寻址方式对页框需求的影响

示例指令分析：

```
MOV [R1+offset1], [R2+offset2]  ; 复杂寻址指令
```

最坏情况页面需求：

1. **指令本身页**：1 页
2. **源操作数页**：1 页 (R2+offset2 指向的页面)
3. **目标操作数页**：1 页 (R1+offset1 指向的页面)
4. **间接寻址页**：若支持多级间接，可能需要更多页

各选项分析

选项	说明	与最少页框数关系	正确性
A. 代码段长	程序代码总长度	影响总页数，不影响并发需求	❌
B. 虚拟地址空间大小	进程可用的虚拟内存范围	影响可访问范围，不影响最少需求	❌
C. 物理地址空间大小	系统物理内存总量	系统限制，不影响进程最少需求	❌
D. 指令系统支持的寻址方式	**ISA 架构的寻址能力**	**直接决定单条指令最大页面需求**	

实际应用举例

不同 ISA 架构的最少页框需求

架构特征	寻址复杂度	典型最少页框数
简单 RISC	寄存器寻址为主	2-3 页
复杂 CISC	多种复杂寻址模式	3-5 页
支持多级间接	间接寻址链	5+页

【答案】D.指令系统支持的寻址方式

确定进程最少页框数的关键是**指令系统的寻址方式**，因为它直接决定了单条指令执行时可能同时需要驻留在内存中的页面数量，这是避免无限缺页中断的基础要求。

28. 关于虚拟文件系统，正确的是（ ）。

A.虚拟文件系统是运行在虚拟内存的文件系统

B.VFS 可以加快文件系统的访问速度

C.VFS 定义了可访问不同文件系统的统一接口

D.VFS 只能访问本地文件系统，不能访问网络文件系统

【答案】C.VFS 定义了可访问不同文件系统的统一接口

VFS 核心概念

虚拟文件系统(Virtual File System)：操作系统内核中的抽象层，为上层应用程序提供统一的文件操作接口，屏蔽底层不同文件系统的实现细节。

VFS 架构图

```
应用程序层
      ↓ (统一系统调用)
VFS 抽象层 ← 核心作用：统一接口
      ↓ (调用具体实现)
具体文件系统 (ext4, NTFS, FAT32, NFS 等)
      ↓
存储设备/网络
```

各选项详细分析

选项	说明	分析结果	正确性
A	VFS 是运行在虚拟内存的文件系统	VFS 是抽象层，不是具体的文件系统实现	❌
B	VFS 可以加快文件系统访问速度	VFS 主要提供抽象，性能提升来自缓存等机制	❌
C	**VFS 定义统一接口访问不同文件系统**	**VFS 的核心功能和设计目标**	
D	VFS 只能访问本地文件系统	VFS 支持网络文件系统如 NFS、SMB 等	❌

VFS 功能特性详解

1. 统一接口抽象

标准系统调用：

- **open(), read(), write(), close()**
- **mkdir(), rmdir(), stat()**
- 应用程序无需关心底层文件系统类型

2. 多文件系统支持

文件系统类型	举例	VFS 支持
本地文件系统	ext4, NTFS, FAT32, XFS	✓ 完全支持
网络文件系统	NFS, SMB/CIFS, AFS	✓ 完全支持
特殊文件系统	procfs, sysfs, tmpfs	✓ 完全支持
分布式文件系统	GFS, HDFS	✓ 通过驱动支持

3. VFS 对象模型

VFS 对象	作用	对应概念
superblock	文件系统元数据	挂载的文件系统实例
inode	文件/目录元信息	文件属性、权限、位置
dentry	目录项缓存	路径名到 inode 的映射
file	打开文件描述	文件操作状态和方法

VFS 工作机制

接口统一示例

```
// 应用程序调用
int fd = open("/mnt/nfs/file.txt", O_RDONLY);   // NFS 文件
int fd2 = open("/home/user/file.txt", O_RDONLY);   // ext4 文件

// VFS 层处理
// 1. 解析路径，确定文件系统类型
// 2. 调用对应文件系统的 open 操作
// 3. 返回统一的文件描述符
```

性能优化机制

虽然 VFS 本身不直接提升性能，但通过以下机制间接优化：

- **dentry 缓存**：路径解析缓存
- **inode 缓存**：文件元数据缓存
- **页面缓存**：文件内容缓存
- **预读机制**：顺序访问优化

实际应用场景

跨平台文件访问

- **Linux**：同时挂载 ext4 和 NTFS 分区
- **网络存储**：透明访问 NFS/SMB 共享
- **容器化**：统一访问宿主机和容器文件系统

开发者优势

- **编程简化**：统一 API，无需适配不同文件系统
- **可移植性**：代码在不同文件系统间无缝运行
- **扩展性**：新文件系统只需实现 VFS 接口

【答案】C.VFS 定义了可访问不同文件系统的统一接口

VFS 的核心价值在于**抽象和统一**，它定义了标准接口，使应用程序能够以相同方式访问各种不同的文件系统，这正是选项 C 所描述的功能。

29. 某文件系统采用索引节点方式。用户在目录中新建文件 F 时，文件系统不会做的是（ ）。
A.初始化文件 F 的索引节点
B.在目录文件中写入 F 的索引节点号
C.在目录文件中写入 F 的访问权限信息
D.在目录文件中增加一条文件 F 对应的目录项
【答案】C.在目录文件中写入 F 的访问权限信息

索引节点(inode)系统架构

索引节点系统：将文件元数据(inode)与目录项(directory entry)分离存储的文件系统设计，典型代表为 Unix/Linux 文件系统。

存储结构对比

组件	存储位置	存储内容	作用
inode	inode 区域	文件元数据(权限、大小、时间戳等)	描述文件属性
目录项	目录文件中	文件名 + inode 号映射	建立名称索引
数据块	数据区域	文件实际内容	存储文件数据

创建文件 F 的详细过程

系统操作步骤表

步骤	操作内容	涉及结构	是否执行
1	**分配空闲 inode**	inode 位图	✓
2	**初始化 inode 结构**	inode 表	✓
3	**在目录中创建目录项**	目录文件	✓
4	**写入文件名和 inode 号**	目录文件	✓

inode 初始化内容

inode 字段	初始化内容	说明
权限位(mode)	根据 umask 设置	文件访问权限
所有者(uid/gid)	当前用户 ID	文件归属
文件大小(size)	0	新文件初始大小
时间戳	当前时间	创建/修改/访问时间
链接计数	1	硬链接数量
数据块指针	空	指向数据块的指针

各选项详细分析

选项	操作描述	实际执行位置	是否执行	正确性
A	初始化文件 F 的索引节点	**inode 表**	✓ 必须执行	正确操作
B	写入 F 的索引节点号	**目录文件**	✓ 必须执行	正确操作
C	**写入 F 的访问权限信息**	**目录文件**	✗ **不会执行**	**错误操作**
D	增加文件 F 的目录项	**目录文件**	✓ 必须执行	正确操作

【答案】C.在目录文件中写入 F 的访问权限信息

文件访问流程

用户访问文件 → 在目录中查找文件名 → 获取 inode 号 → 读取 inode → 检查权限 → 访问数据块

在索引节点文件系统中，**权限信息存储在 inode 中而非目录项中**。创建文件时，系统会初始化 inode、在目录中添加目录项记录文件名和 inode 号的映射，但**不会在目录文件中写入访问权限信息**。

30.关于内存映射文件，下列说法正确的是（ ）。

I.可实现进程间通信

II.实现了页面到磁盘块的映射

III.将文件映射到进程的虚拟地址空间

IV.将文件映射到系统的物理地址空间

A.I、III

B.I、IV

C.II、III

D.I、II、III

【答案】A.I、III

内存映射文件（Memory-Mapped File）是操作系统提供的一种文件 I/O 机制，通过虚拟内存管理单元（MMU）将文件内容直接映射到进程的虚拟地址空间，实现文件访问的零拷贝操作。

选项分析

选项	描述	正确性	技术分析
I	可实现进程间通信		多个进程通过 mmap()系统调用映射同一文件，形成共享内存区域，支持 IPC 机制
II	实现页面到磁盘块的映射	✗	混淆了抽象层次：映射的是虚拟页面到文件偏移量，磁盘块映射由文件系统管理
III	映射到进程虚拟地址空间		核心机制：在进程 VMA（Virtual Memory Area）中建立文件到虚拟地址的映射关系
IV	映射到物理地址空间	✗	违背虚拟内存原理：映射目标是虚拟地址，物理地址由页表和 MMU 负责转换

技术实现机制

虚拟内存映射层次：

- **应用层**：进程通过虚拟地址访问文件内容
- **内核层**：VMA 结构维护映射关系，页面错误处理器负责按需加载
- **硬件层**：MMU 执行虚拟到物理地址转换

IPC 实现原理： 当多个进程映射同一文件时，操作系统确保对应的物理页面在内存中共享，修改操作通过写时拷贝（Copy-on-Write）或直接共享机制同步到文件。

【答案】A.I、III

选项 I 和 III 准确描述了内存映射文件的核心特性和实现机制。选项 II 错误地将文件系统的磁盘块管理与虚拟内存映射混淆；选项 IV 违背了现代操作系统的虚拟内存设计原则。

31.下列选项中，文件系统能够知道外存空闲空间的使用情况是（ ）。
A.目录 B.系统文件打开表 C.文件分配表（FAT） D.文件控制块（FCB）【答案】C.文件分配表（FAT）

外存空闲空间管理概念

文件系统必须维护**全局存储空间使用状态**，包括：

- 哪些块/簇已被占用
- 哪些块/簇处于空闲状态
- 如何快速分配和回收存储空间

各选项功能对比

选项	主要功能	作用范围	空闲空间管理	正确性
A. 目录	文件名→元数据映射	单个目录	❌不涉及	❌
B. 系统文件打开表	跟踪已打开文件状态	当前会话	❌ 不涉及	❌
C. 文件分配表(FAT)	**记录所有簇的使用状态**	**整个磁盘**	**核心功能**	
D. 文件控制块(FCB)	单个文件属性描述	单个文件	❌不涉及	❌

FAT 文件系统详细分析

FAT 表结构与功能

FAT 表项值	含义	空间状态
0x000	空闲簇	**可分配**
0x002-0xFEF	下一簇号(链接)	已占用
0xFF0-0xFF6	保留簇	已占用
0xFF7	坏簇标记	不可用
0xFF8-0xFFF	文件结束标记	已占用

空闲空间识别机制

```
// FAT 空闲簇查找示例
for (int cluster = 2; cluster < total_clusters; cluster++) {
    if (FAT[cluster] == 0x000) {
        // 发现空闲簇，可以分配
        return cluster;
    }
}
```

其他文件系统空闲空间管理方法

常见管理策略对比

文件系统	空闲空间管理方法	实现机制
FAT	FAT 表标记	表项为 0 表示空闲
ext2/3/4	空闲块位图	位图中 0 表示空闲
NTFS	Bitmap 文件	专用位图文件
Unix FS	空闲块链表	超级块维护空闲链

各方法优缺点

方法	优点	缺点	适用场景
FAT 表	简单直观，查找便捷	表大小固定，大磁盘效率低	小容量存储
位图	空间效率高，随机访问快	需额外位图空间	现代文件系统
链表	动态管理，节省空间	顺序访问，性能较低	传统 Unix 系统

【答案】C.文件分配表（FAT）

32.下列选项中，文件系统能为温彻斯特硬盘和固态硬盘提供的功能是（ ）。

A.划分扇区　B.确定盘块大小　C.降低寻道时间　D.实现均衡磨损

【答案】B.确定盘块大小

温彻斯特硬盘（Winchester Disk）是现代机械硬盘（HDD）的核心技术原型，其名称源于 IBM 在 1973 年推出的首款采用该技术的硬盘 IBM 3340。这一技术通过密封结构、悬浮磁头和高速旋转盘片，彻底改变了数据存储的可靠性和容量，成为当今硬盘的基础。 **HDD vs SSD 基本特性**

特性	机械硬盘(HDD)	固态硬盘(SSD)
存储介质	磁性盘片	NAND 闪存
访问方式	磁头寻道+旋转	电子访问
寻道时间	存在(3-15ms)	不存在
磨损特性	机械磨损	擦写次数限制
随机访问	性能较差	性能优异

各选项详细分析

选项	功能描述	实现层次	HDD 适用	SSD 适用	通用性	正确性
A	划分扇区	硬件/分区层			❌ 非文件系统功能	❌
B	**确定盘块大小**	**文件系统层**			✅**通用抽象**	
C	降低寻道时间	硬件/调度层		❌不存在寻道	❌ 非通用功能	❌
D	实现均衡磨损	控制器层	❌ 不需要		❌ SSD 特有	❌

盘块大小确定机制

文件系统抽象层功能

盘块(Block/Cluster)：文件系统进行存储管理的基本单位

文件系统	默认块大小	可配置范围	底层适配
ext4	4KB	1KB-64KB	适配任何块设备
NTFS	4KB	512B-64KB	适配 HDD/SSD
FAT32	4KB-32KB	依据容量调整	硬件无关
XFS	4KB	512B-64KB	通用块设备

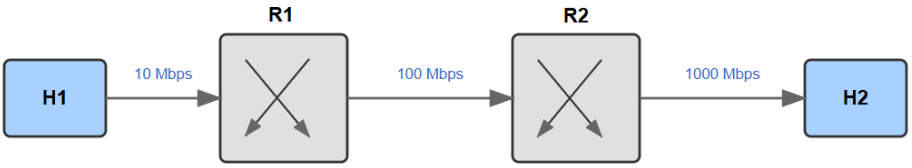
块大小选择考虑因素

因素	小块大小	大块大小	设备影响
空间利用率	高(减少内部碎片)	低	HDD/SSD 通用
I/O 效率	低(频繁访问)	高	HDD/SSD 通用
文件类型适应	小文件友好	大文件友好	与设备类型无关

【答案】B.确定盘块大小

文件系统的核心职责是提供**设备无关的存储抽象**。确定盘块大小是文件系统在格式化时执行的基本功能，它为 HDD 和 SSD 提供统一的逻辑块管理机制，屏蔽底层硬件差异，这是唯一对两种设备都适用的通用文件系统功能。

33.如下图所示，主机 H1 向 H2 发送一个 2MB（1M=10^6B）文件有三种方式：



①电路交换，建立时间为 32us，速度为 10Mbps；②分组交换，文件长度为 400B，忽略首部；③报文交换。电路交换的时间为 T_{cs}，报文交换的时间为 T_{ms}，分组交换的时间为 T_{ps}，则三者的大小关系是（ ）。

A. $T_{cs} > T_{ms} > T_{ps}$

B. $T_{ms} > T_{ps} > T_{cs}$

C. $T_{ms} > T_{cs} > T_{ps}$

D. $T_{ps} > T_{ms} > T_{cs}$

【答案】B. $T_{ms} > T_{ps} > T_{cs}$基

本参数

参数	数值	说明
文件大小	2MB = 2×10^6B = 16×10^6bit	待传输文件
分组大小	400B (忽略首部)	分组交换参数
电路建立时间	32μs	电路交换建立开销

网络链路参数

链路	带宽	传输速率
H1→R1	10 Mbps	瓶颈链路
R1→R2	100 Mbps	中间链路
R2→H2	1000 Mbps	高速链路

三种交换方式时延计算

1. 电路交换 (Tcs)

特点：建立连接后端到端传输，受瓶颈链路限制

阶段	时间计算	结果
连接建立	32μs	0.000032s
数据传输	16×10^6bit ÷ 10×10^6bps	1.6s
总时间	0.000032 + 1.6	**Tcs ≈ 1.600032s**

2. 分组交换 (Tps)

特点：流水线传输，首包穿越时延 + 剩余包受瓶颈限制

分组划分

分组数量 = 2×10^6B ÷ 400B = 5000 个分组

单分组传输时延

链路	传输时延计算	时延值
H1→R1 (10Mbps)	400B×8bit ÷ 10×10^6bps	0.32ms
R1→R2 (100Mbps)	400B×8bit ÷ 100×10^6bps	0.032ms
R2→H2 (1000Mbps)	400B×8bit ÷ 1000×10^6bps	0.0032ms

流水线传输分析

首包穿越时延 = 0.32 + 0.032 + 0.0032 = 0.3552ms
剩余 4999 包受瓶颈限制，每包间隔 = 0.32ms
总时延 = 0.3552 + 4999×0.32 = 1600.0352ms
Tps ≈ 1.6000352s

3. 报文交换 (Tms)

特点: 整个报文逐跳存储转发，无流水线效应

逐跳传输时延

跳数	链路	传输时延计算	时延值
第 1 跳	H1→R1 (10Mbps)	16×10^6bit ÷ 10×10^6bps	1.6s
第 2 跳	R1→R2 (100Mbps)	16×10^6bit ÷ 100×10^6bps	0.16s
第 3 跳	R2→H2 (1000Mbps)	16×10^6bit ÷ 1000×10^6bps	0.016s

总时延 = 1.6 + 0.16 + 0.016 = 1.776s

Tms = 1.776s

三种方式对比分析

时延比较表

交换方式	计算结果	主要特征	优缺点
电路交换	**Tcs ≈ 1.600032s**	建立后连续传输	建立开销小，传输效率高
分组交换	**Tps ≈ 1.6000352s**	流水线传输	轻微额外时延，资源利用好
报文交换	**Tms = 1.776s**	逐跳存储转发	时延最大，占用存储多

【答案】B. $T_{ms} > T_{ps} > T_{cs}$

瓶颈链路影响

交换方式	瓶颈影响程度	说明
电路交换	完全受限	端到端传输受最慢链路限制
分组交换	主要受限	流水线稳态受瓶颈控制
报文交换	累积受限	每跳都受该跳链路限制

文件大小敏感性

- **电路交换**: 对大文件友好，建立开销摊薄
- **分组交换**: 分组数量影响流水线效率
- **报文交换**: 大文件在每跳都造成长时延

34. 某差错编码的编码集为 {1001 1010, 0101 1100, 1111 0000, 0000 1111}，它的检错和纠错能力是（ ）。

A.可检测不超过 2 位错，检错率 100%；可纠正不超过 1 位错

B.可检测不超过 2 位错，检错率 100%；可纠正不超过 2 位错

C.可检测不超过 3 位错，检错率 100%；可纠正不超过 1 位错

D.可检测不超过 3 位错，检错率 100%；可纠正不超过 2 位错

【答案】C.可检测不超过 3 位错，检错率 100%；可纠正不超过 1 位错

1. 计算汉明距离

汉明距离是两个等长字符串对应位置不同字符的个数。

码字对	码字 1	码字 2	汉明距离计算	距离值
1	10011010	01011100	第 1,2,6,7 位不同	4
2	10011010	11110000	第 2,3,5,7 位不同	4
3	10011010	00001111	第 1,4,6,8 位不同	4
4	01011100	11110000	第 1,3,5,6 位不同	4
5	01011100	00001111	第 2,4,7,8 位不同	4
6	11110000	00001111	全部 8 位不同	8

最小汉明距离 d_min = 4

2. 检错和纠错能力判断

根据编码理论：

- **检错能力**：可检测不超过 (d_min - 1) 位错，检错率 100%
- **纠错能力**：可纠正不超过 ⌊(d_min - 1)/2⌋ 位错

3. 计算结果

- 检错能力：d_min - 1 = 4 - 1 = 3 位错
- 纠错能力：⌊(4-1)/2⌋ = ⌊3/2⌋ = 1 位错

【答案】C.可检测不超过 3 位错，检错率 100%；可纠正不超过 1 位错

该编码集的能力为：

- 可检测不超过 3 位错，检错率 100%
- 可纠正不超过 1 位错

35. 现有一 10BaseT 以太网，甲乙处于同一个冲突域，连续发生 11 次冲突，甲再次发送的最大时间间隔为（ ）。

A.0.512ms

B.0.5632ms

C.52.3776ms

D.104.8064ms

【答案】C.52.3776ms

二进制指数退避算法

- 连续第 n 次冲突后，随机选择 $r \in [0, 2^k - 1]$个时隙等待
- 其中：k = min(n, 10)
- 10BaseT 槽时间 = 512 bit-times = 51.2 μs

计算步骤

1. 确定退避参数

参数	数值	说明
冲突次数 n	11	连续发生 11 次冲突
退避指数 k	min(11,10) = 10	最大为 10
随机数范围	$[0, 2^{10} - 1]$	[0, 1023]
最大随机数 r_max	1023	求最大时间间隔

2. 计算最大等待时间

最大时间间隔 = r_max × 槽时间

最大时间间隔 = 1023 × 51.2 μs = 52,377.6 μs = **52.3776 ms**

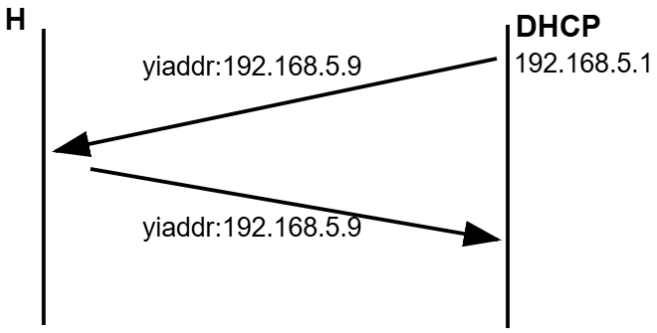
各选项分析

选项	数值	对应随机数	是否正确
A	0.512ms	r=10	❌ 非最大值
B	0.5632ms	r=11	❌ 非最大值
C	**52.3776ms**	**r=1023**	**最大值**
D	104.8064ms	r=2046	❌ 超出范围

【答案】C.52.3776ms

连续 11 次冲突后，甲再次发送的最大时间间隔为 52.3776ms。

36.一台新接入网络的主机 H 通过 DHCP 服务器动态请求 IP 地址过程中，与 DHCP 服务器交换 DHCP 报文过程如下图所示。封装 DHCP 的 REQUEST 报文的 IP 数据报的目的 IP 地址和源 IP 地址分别是（ ）。



A. 192.168.5.1, 0.0.0.0

B. 192.168.5.1, 192.168.5.9

C. 255.255.255.255,0.0.0.0

D. 255.255.255.255, 192.168.5.9

【答案】C. 255.255.255.255,0.0.0.0

DHCP 工作原理

新接入主机通过 4 步完成 IP 地址获取：DISCOVER → OFFER → REQUEST → ACK **REQUEST 报文 IP 地址分析**

地址类型	使用地址	原因说明
源 IP 地址	**0.0.0.0**	主机尚未获得正式 IP 地址
目的 IP 地址	**255.255.255.255**	广播发送，确保 DHCP 服务器可达

【答案】C. 255.255.255.255,0.0.0.0

关键要点

- 主机在获得 IP 前必须使用 0.0.0.0 作为源地址
- REQUEST 报文采用广播形式，避免中间设备过滤
- 只有在续租等特殊情况下才使用单播

37. 假设路由器实现 NAT 功能，内网中主机 H 的 IP 地址为 192.168.1.5/24，若 H 运行某应用向 internet 发送一个 UDP 报文段，则路由器在转发封装该 UDP 报文段的 IP 数据报的过程中，UDP 报文的首部字段会被修改的是（ ）。

Ⅰ 源端口号 Ⅱ 目的端口号 Ⅲ 总长度 Ⅳ 校验和

A. Ⅰ、Ⅲ

B. Ⅰ、Ⅳ

C. Ⅱ、Ⅲ

D. Ⅱ、Ⅳ

【答案】B. Ⅰ、Ⅳ

NAT 工作机制

NAT 路由器将内网 IP 映射为公网 IP，可能修改端口号实现多路复用。

UDP 字段修改分析

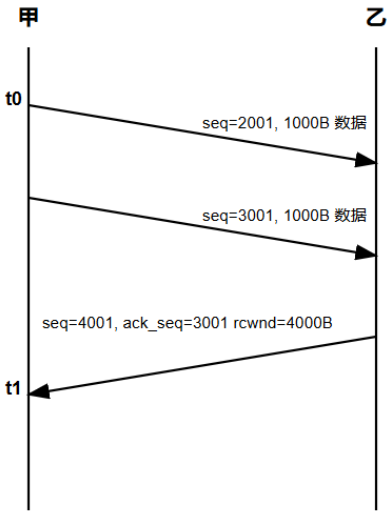
字段	是否修改	修改原因
Ⅰ 源端口号	✓ 修改	NAT 端口复用，区分不同内网主机
Ⅱ 目的端口号	✗ 不变	外网服务端口通常不改
Ⅲ 总长度	✗ 不变	UDP 载荷数据未改变
Ⅳ 校验和	✓ 修改	源 IP/源端口变化需重算校验和

【答案】B. Ⅰ、Ⅳ

UDP 校验和计算

- 覆盖伪首部（源/目的 IP、协议号、UDP 长度）+ UDP 首部和数据
- 源 IP 或源端口改变 → 必须重新计算校验和

38. 主机甲通过 TCP 向主机乙发送数据的部分过程如下图，seq 为序号，ack-seq 为确认序号，rewnd 为接收窗口。甲在 t0 时刻的拥塞窗口和发送窗口均为 2000B，拥塞控制阈值为 8000B，MSS=1000B。甲始终以 MSS 发送 TCP 段。若甲在 t1 时刻收到如图所示的确认段，则甲在未收到新的确认段之前，还可以继续向乙发送的 TCP 段数是（ ）。



A.2

B.3

C.4

D.5

【答案】A.2

初始状态分析

参数	数值	说明
t0 时刻 cwnd	2000B	拥塞窗口
t0 时刻发送窗口	2000B	min(cwnd, rwnd)
ssthresh	8000B	慢启动阈值
MSS	1000B	最大分段大小

t0→t1 过程分析

1.t0 发送： 甲发送 2 个 MSS 段

段 1：seq=2001～3000 (1000B)

段 2：seq=3001～4000 (1000B)

在途数据：2000B

2.t1 收到 ACK： ack_seq=3001, rwnd=4000B

累计确认到 seq=3000（段 1 已确认）

在途数据：1000B（段 2 未确认）

3.拥塞控制更新

cwnd < ssthresh（慢启动阶段）

cwnd += MSS = 2000 + 1000 = 3000B

可发送计算

1.发送窗口 = min(cwnd, rwnd) = min(3000, 4000) = 3000B

2.可发送 = 发送窗口 - 在途数据 = 3000 - 1000 = 2000B

3.可发送段数 = 2000B ÷ 1000B = **2 段**

【答案】A.2

39. Time 是一个提供时间查询服务的 C/S 架构网络应用，支持客户通过 UDP 和 TCP 向Time 服务器请求时间。若某客户与 Time 服务器通信往返时间为 8ms，则该客户分别通过UDP 和 TCP 向该服务器请求服务，所需的最少时间分别是（ ）。

A.8ms，8ms

B.8ms，16ms

C.16ms，8ms

D.16ms，16ms

【答案】B.8ms，16ms

协议特点对比

协议	连接特性	请求过程	所需 RTT
UDP	无连接	直接请求-响应	1 个 RTT
TCP	面向连接	握手+请求-响应	2 个 RTT

时间计算

- **UDP**：请求→响应 = 1 RTT = 8ms
- **TCP**：三次握手(1 RTT) + 请求响应(1 RTT) = 2 RTT = 16ms

【答案】B.8ms，16ms

40. 下列关于 POP3 协议的叙述，正确的是

Ⅰ. 支持用户代理从邮件服务器读取邮件

Ⅱ. 支持用户代理向邮件服务器发送邮件

Ⅲ. 支持邮件服务器之间发送与接收邮件

Ⅳ. 支持一条 TCP 连接接收多封邮件

A. Ⅰ、Ⅳ

B. Ⅱ、Ⅲ

C. Ⅰ、Ⅱ、Ⅲ

D. Ⅰ、Ⅲ、Ⅳ

【答案】A. Ⅰ、Ⅳ **邮件系统协议分工**

协议	主要功能	工作场景
POP3	邮件读取/下载	用户代理←→邮件服务器
SMTP	邮件发送/转发	用户代理→邮件服务器，服务器间传输
IMAP	邮件管理	用户代理←→邮件服务器

各选项分析

选项	描述	正确性	说明
Ⅰ	支持从服务器读取邮件	✅正确	POP3 核心功能
Ⅱ	支持向服务器发送邮件	❌错误	发送邮件由 SMTP 负责
Ⅲ	支持服务器间邮件传输	❌错误	服务器间传输由 SMTP 负责
Ⅳ	支持一条 TCP 连接接收多封邮件	✅正确	可多次 RETR 命令

【答案】A. Ⅰ、Ⅳ

41【13 分】设有两个长度均为 n 的一维整型数组 A 和 res，对数组 A 中的每个元素 A[i]，计算 A[i]与 A[j]（0<=i<=j<=n-1）乘积的最大值，并将其保存到 res[i]中。例如，若 A[]={1，4，-9，6)，则得到 res[]=（6，24，81，36)。现给定数组 A，请设计一个时间和空间上尽可能高效的算法 calMulMax，求 res 中各元素的值。

函数原型为：void calMulMax（intA[]，int res[]，int n)，

要求：

（1）给出算法的基本设计思想；（4 分）

（2）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释；（7 分）

（3）说明你所设计算法的时间复杂度和空间复杂度。（2 分）

（1）基本设计思想（4 分）

核心思路

对于固定的 A[i]，要使 A[i]×A[j]最大，需根据 A[i]的符号选择后缀中的极值：

A[i]的符号	选择策略	原因
A[i] ≥ 0	选择后缀最大值	正数×最大数=最大乘积
A[i] < 0	选择后缀最小值	负数×最小数=最大乘积

算法策略

- **从右向左扫描**：动态维护当前位置及右侧的后缀最小值和最大值
- **一次遍历**：每个位置根据符号选择对应极值计算结果

（2）算法实现（7 分）

```
void calMulMax(int A[], int res[], int n) {
    if (n <= 0) return;   // 边界处理
    // 维护后缀最小值和最大值
    int minSuffix = INT_MAX;
    int maxSuffix = INT_MIN;
    // 从右到左扫描，确保后缀极值包含 A[i]自身
    for (int i = n - 1; i >= 0; --i) {
        // 将当前元素纳入后缀极值计算
        if (A[i] < minSuffix) minSuffix = A[i];
        if (A[i] > maxSuffix) maxSuffix = A[i];

        // 根据 A[i]符号选择后缀极值
        if (A[i] >= 0) {
            res[i] = A[i] * maxSuffix;   // 非负数选最大值
        } else {
            res[i] = A[i] * minSuffix;   // 负数选最小值
        }
    }
}
```

关键注释说明

- **后缀极值更新在前**：保证 j 可以等于 i（满足 i≤j 条件）
- **符号判断**：A[i]=0 时归为非负，结果必为 0
- **负数策略**：选择最小值可能是负数（得正积）或 0（得 0）

（3）复杂度分析（2 分）

复杂度类型	结果	说明
时间复杂度	**O(n)**	单次线性扫描
空间复杂度	**O(1)**	仅用两个辅助变量

皮皮灰不同算法方案评分标准

算法思路评分（4 分）

得分	算法思路	特点
4 分	后缀极值+符号判断	O(n)时间，O(1)空间
3 分	预处理后缀数组	O(n)时间，O(n)空间
2 分	双重循环优化	$O(n^2)$时间，部分剪枝
1 分	暴力双重循环	$O(n^2)$时间，无优化
0 分	思路错误或缺失	逻辑错误

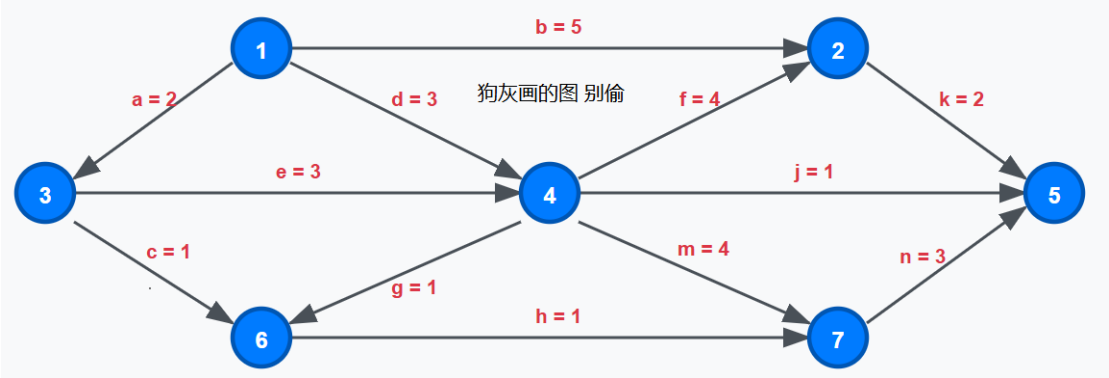
代码实现评分（7 分）

得分	实现质量	要求
6-7 分	完整正确实现	逻辑正确，边界处理完善，注释清晰
4-5 分	基本正确	主体逻辑对，少量细节问题
2-3 分	部分正确	思路对但实现有明显错误
1 分	框架正确	有基本结构但逻辑错误较多
0 分	错误实现	无法运行或逻辑完全错误

复杂度分析评分（2 分）

得分	分析准确性
2 分	时间和空间复杂度都正确
1 分	仅一个复杂度正确
0 分	复杂度分析错误或缺失

42【10 分】某工程包含 12 活动，使用题 42 图所示的 AOE 网描述，图中各边上标注了活动及其持续时间。



请回答下列问题（活动均用活动名表示）

（1）完成该工程的最短时间是多少？哪些活动是关键活动？（3 分）

（2）若按照最短时间完成工程，则与活动 e 同时进行的活动可能有哪些？（3 分）

（3）时间余量最大的活动是哪个？其时间余量是多少？（2 分）

（4）假设工程从时刻 0 启动，因某种原因，活动 b 在时刻 6 开始，为保证工程不延期，在其他活动持续时间不变的情况下，b 的持续时间最多是多少？若不改变 b 的持续时间，则压缩哪个活动的持续时间也能保证工程不延期？（2 分）

（1）最短完成时间与关键活动（3 分）

事件最早发生时间 ve(i)和最迟发生时间 vl(i)

节点	ve(i)	vl(i)	计算过程
1	0	0	起点
2	9	10	max(0+5, 5+4) = 9
3	2	2	0+2 = 2
4	5	5	max(0+3, 2+3) = 5
5	12	12	max(9+2, 9+3) = 12
6	6	8	max(2+1, 5+1) = 6
7	9	9	max(6+1, 5+4) = 9

活动时间参数计算

活动	ES	EF	LS	LF	余量	是否关键
a(1→3)	0	2	0	2	0	是
b(1→2)	0	5	5	10	5	否
d(1→4)	0	3	2	5	2	否
e(3→4)	2	5	2	5	0	是
c(3→6)	2	3	7	8	5	否
f(4→2)	5	9	6	10	1	否
g(4→6)	5	6	7	8	2	否
j(4→5)	5	6	11	12	6	否
m(4→7)	5	9	5	9	0	是
h(6→7)	6	7	8	9	2	否

活动	ES	EF	LS	LF	余量	是否关键
k(2→5)	9	11	10	12	1	否
n(7→5)	9	12	9	12	0	是

答案：

最短完成时间：12

关键活动：a, e, m, n

关键路径：1→3→4→7→5

（2）与活动 e 同时进行的活动（3 分）

活动 e 的执行时间窗口：[2, 5]

在最优调度下，可能与 e 同时进行的活动：

活动	最早开始	最晚结束	时间窗口	与 e 重叠
b	0	10	[0,5]	**是**
d	0	5	[2,5]	**是**
c	2	8	[7,8]	**是**（可调整到[2,3]）

答案：b, d, c

（3）时间余量最大的活动（2 分）

活动	时间余量
j	6（最大）
b	5
c	5
d	2
g	2
h	2
f	1
k	1

答案：活动 j，时间余量为 6

（4）活动 b 延期的影响分析（2 分）

b 在时刻 6 开始的约束分析

- 原始 b：[0,5]，现在 b：[6, 6+tb]
- 节点 2 的发生时间：max(6+tb, 9)（受 f 活动约束）
- 通过路径 2→5：max(6+tb, 9) + 2 ≤ 12

计算过程：

max(6+tb, 9) + 2 ≤ 12

当 6+tb ≥ 9 时，即 tb ≥ 3 时：

6+tb + 2 ≤ 12 → tb ≤ 4

压缩方案分析

若 b 持续时间仍为 5：

- 节点 2 发生时间：max(6+5, 9) = 11
- k 活动必须在时刻 11 完成，需压缩 k 从 2→1

答案：

b 的持续时间最多为 4

可压缩活动 k 的持续时间（从 2 压缩到 1）

no.【12 分】现有 C 语言程序 P 的部分代码如题 43 图所示，假定运行程序 P 的计算机 M 字长为 32 位，按字节编址，数据 Cache 的数据区大小为 32KB，采用 8 路组相联映射方式，主存块大小为 64B，Cache 命中时间为 2 个时钟周期，缺失损失为 200 个时钟周期：采用页式虚拟存储管理方式，页大小为 4kB，数组 d 起始虚拟地址为 0180 0020H（VA31~VA0）

```
int x, d[2048];
…
for(i=0; i<2048;i++)
        d[i]=d[i]/x;
…
```

题 43 图 程序 P 的部分代码

请回答下列问题：

（1）主存地址中 cache 组号字段和块内地址字段分别占几位？虚拟地址中哪些位可作为 cache 索引？（2 分）

（2）d[100]的虚拟地址为多少？d[100]所在主存块对应的 cache 组号是多少？（3 分）

（3）假设执行 for 语句时对应代码已在 cache，变量 i 和 x 已装入寄存器，数组 d 已调入主存但不在 cache，则 d[0]在其所在主存块内的偏移量是多少（用十六进制），for 语句执行过程中，访问数组 d 的 cache 缺失率和数组元素的平均访问时长是多少？（缺失率百分比保留两位）（4 分）

（4）数组 d 分布在几个页中？若执行 for 语句时对应代码已经在主存，但数组 d 还未调入主存，则执行 for 语句过程中，访问数组 d 所引起的缺页次数是多少？（3 分）

系统参数总结

系统组件	参数配置	数值
Cache	数据区大小	32KB
	映射方式	8 路组相联
	主存块大小	64B
	命中时间	2 个时钟周期
	缺失损失	200 个时钟周期
虚拟存储	页大小	4KB
	数组起始地址	0x01800020
数据类型	int 大小	4 字节
	数组大小	2048 个元素

问题(1)：地址字段位数分析

Cache 参数计算

Cache 总块数 = 32KB ÷ 64B = 512 块
组数 = 512 块 ÷ 8 路 = 64 组

地址字段划分

字段类型	位数	计算依据
块内地址	6 位	$\log_2(64B) = 6$ 位
组号字段	6 位	$\log_2(64 组) = 6$ 位
标记字段	20 位	32 - 6 - 6 = 20 位

Cache 索引位确定

- **页大小**：4KB = 2^{12}，页内偏移 12 位(VA_{11}-VA_0)
- **Cache 索引**：使用 VA_{11}-VA_6(6 位组号字段)
- **块内偏移**：使用 VA_5-VA_0(6 位块内地址)

问题(2)：d[100]地址计算

虚拟地址计算

d[100]地址 = 起始地址 + 100 × sizeof(int)
= 0x01800020 + 100 × 4
= 0x01800020 + 0x190
= 0x018001B0

Cache 组号计算

组号 = (地址 & 0xFC0) >> 6
= (0x018001B0 & 0xFC0) >> 6
= 0x180 >> 6
= 6

问题(3)：Cache 性能分析

d[0]块内偏移量

d[0]地址 = 0x01800020
块内偏移 = 0x01800020 & 0x3F = 0x20

主存块数计算

- **数组总大小**：2048 × 4 = 8192 字节
- **首块可用空间**：64 - 32 = 32 字节
- **剩余数据**：8192 - 32 = 8160 字节
- **额外块数**：⌈8160 ÷ 64⌉ = 128 块
- **总块数**：1 + 128 = 129 块

Cache 缺失率与平均访问时间

访问模式	数值计算
总访问次数	2048 × 2 = 4096 次(读+写)
缺失次数	129 次(每块首次读缺失)
缺失率	129 ÷ 4096 ≈ 3.15%
平均访问时间	2 + 0.0315 × 200 ≈ 8.30 周期

问题(4)：页分布与缺页分析

数组页分布计算

起始地址：0x01800020 (页内偏移：0x020 = 32 字节)
第 1 页剩余：4096 - 32 = 4064 字节 (1016 个 int)
已用数据：1016 × 4 = 4064 字节
剩余数据：8192 - 4064 = 4128 字节

第 2 页：完整 4096 字节 (1024 个 int)
剩余数据：4128 - 4096 = 32 字节

第 3 页：32 字节 (8 个 int)

页分布与缺页统计

页号	存储元素	数据量	缺页次数
第 1 页	d[0]-d[1015]	4064B	1 次
第 2 页	d[1016]-d[2039]	4096B	1 次
第 3 页	d[2040]-d[2047]	32B	1 次
总计	2048 个元素	8192B	**3 次**

关键知识点补充

Cache 映射方式对比

映射方式	硬件复杂度	命中率	冲突概率
直接映射	低	低	高
组相联	中等	中等	中等
全相联	高	高	低

虚拟地址翻译流程

1. **页表查询**：VA[31:12] → 物理页号
2. **地址拼接**：物理页号 + VA[11:0]
3. **Cache 索引**：使用 VA[11:6]进行 Cache 访问

答案总结

(1) 组号字段：6 位，块内地址字段：6 位，Cache 索引使用 VA_{11}-VA_6

(2) d[100]虚拟地址：0x018001B0，对应 Cache 组号：6

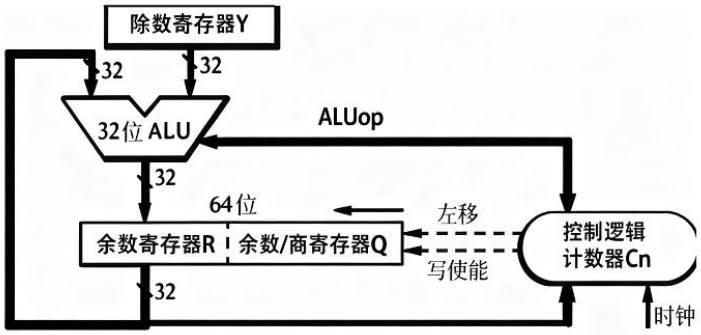
(3) d[0]块内偏移：0x20，缺失率：3.15%，平均访问时长：8.30 周期

(4) 数组 d 分布在 3 个页中，缺页次数：3 次

44.【9 分】对于题 43 中计算机 M 和程序 P，假定 P 的部分机器级代码如题 44（a）图所示，其中，R0-R4 为通用寄存器，SEXT 表示按符号扩展；M 中补码除法器逻辑结构如题 44（b）图所示。

```
…
//X 在 R2 中，i 在 R4 中
//数组 d 的首地址在 R3 中
mov R1，（R3+R4*4）     //R1 ←d[i]
scov R1                 //（R0，R1）←SEXT（R1）
idiv R1                 //R1 ←（（R0，R1）/R2）
…
```

题 44（a）图 程序代码



题 44（b）图 补码除法器逻辑结构

请回答下列问题，并说明理由。

（1）若执行题 44（a）图中 idiv 指令的除运算时，d[i]=0x87654321，x=0xff，则补码除法器中寄存器 R、Q 和 Y 的初始内容分别是什么（用十六进制表示）？题 44（b）图中哪个部件包含计数器？在补码除法器执行过程中，由 ALUop 所控制的 ALU 运算有哪几种？（4 分）

（2）假设 idiv 指令执行过程中会检测并触发除法异常，则执行 idiv 指令时，哪些情况下会发生除法异常（要求给出此时 d[i]和 x 的十六进制表示机器数）？发生除法异常时，在异常响应过程中 CPU 需要完成哪些操作？（5 分）

指令执行过程分析

指令序列执行流程

指令	操作	结果
mov R1,(R3+R4*4)	加载 d[i]到 R1	R1 ← d[i]
scov R1	符号扩展到 64 位	(R0,R1) ← SEXT(R1)
idiv R1	有符号除法	R1 ← (R0,R1)/R2

问题(1)：除法器初始状态

数据准备过程

```
d[i] = 0x87654321 (32 位负数)
x = 0x000000FF (32 位正数)
```

符号扩展结果

```
scov R1 执行后：
R0 = 0xFFFFFFFF (高 32 位，符号扩展)
```

R1 = 0x87654321 (低 32 位，原值)
R2 = 0x000000FF (除数 x)

补码除法器寄存器初始值

寄存器	初始内容	说明
R (余数寄存器)	0xFFFFFFFF	被除数高 32 位
Q (商寄存器)	0x87654321	被除数低 32 位
Y (除数寄存器)	0x000000FF	除数

控制部件与 ALU 操作

组件	功能	包含计数器
控制逻辑 Cn	循环控制、信号发生	包含计数器
ALU	算术逻辑运算	

ALUop 控制的运算类型：

- **R - Y**：试探性减法
- **R + Y**：恢复性加法
- **传送操作**：数据移动
- **比较判断**：符号位检测

1. 恢复余数法中，先执行 R-Y 试探余数符号；
2. 若余数为负，需执行 R+Y 恢复余数；
3. 传送操作用于更新寄存器内容，比较判断用于检测余数符号（决定后续操作）。

问题(2)：除法异常分析

除法异常触发条件

条件 1：除数为零

参数	十六进制值	说明
d[i]	任意值(如 0x87654321)	被除数
x	0x00000000	除数为 0

条件 2：商溢出

参数	十六进制值	数学含义
d[i]	0x80000000	-2^{31} (INT_MIN)
x	0xFFFFFFFF	-1
商	$+2^{31}$	超出 32 位有符号范围

32 位有符号数范围

有效范围：$-2^{31} \le$ 商 $\le 2^{31}-1$
即：0x80000000 ≤ 商 ≤ 0x7FFFFFFF

异常响应机制

CPU 异常处理流程

阶段	操作内容	目的
1. 异常检测	识别异常类型和原因	确定处理方式
2. 现场保护	保存 PC、状态寄存器等	支持异常返回
3. 状态切换	进入内核态，屏蔽中断	保证处理安全
4. 向量跳转	根据异常类型跳转处理程序	执行异常处理
5. 流水线控制	冲刷未完成指令	保证执行一致性

具体操作步骤

1. 记录异常信息
   - 异常类型 → 异常原因寄存器
   - 异常 PC → EPC 寄存器

2. 保存执行现场
   - 当前 PC 值
   - 处理器状态字(PSW)
   - 必要的通用寄存器

3. 系统状态切换
   - 用户态 → 内核态
   - 用户栈 → 内核栈
   - 中断屏蔽设置

4. 异常处理跳转
   - 查询异常向量表
   - 跳转到对应处理程序
   - 流水线冲刷/回滚

5. 异常处理完成
   - 执行 RFE/ERET 指令返回
   - 或向进程发送信号/终止进程

补充知识点

补码除法算法特点

算法类型	特点	操作方式
恢复余数法	试探后恢复	减法→判断→可能加回
不恢复余数法	直接调整	根据余数符号选择加/减

异常与中断区别

类型	触发源	时机	处理特点
异常	指令执行	同步	与指令相关
中断	外部事件	异步	与指令无关

答案总结 (1)

- 寄存器初始内容：R=0xFFFFFFFF，Q=0x87654321，Y=0x000000FF
- 控制逻辑 Cn 包含计数器
- ALU 运算：R-Y(减法)、R+Y(加法)（传送、比较判断）

(2)

- 异常条件：①除数为 0 (x=0x00000000)；②商溢出 (d[i]=0x80000000，x=0xFFFFFFFF)
- CPU 操作：异常检测→现场保护→状态切换→向量跳转→流水线控制→异常处理

45.【8 分】三个人一起植树，甲挖坑，乙放树苗入坑并填土，丙负责为新种树苗浇水。步骤依次为：挖树坑，放树苗，填土和浇水。现在有铁锹和水桶各一个，铁锹用于挖树坑，填土。水桶用于浇水。当树坑数量小于 3 时，甲才可以挖树坑。设初始坑=0，铁锹水桶均可用，定义尽可能少的信号量，用 wait()和 signal()。操作描述植树过程中三人的同步互斥关系，并说明所用信号量的作用及其初值。

约束类型	具体要求	实现方式
流程顺序	挖坑→放树苗→填土→浇水	同步信号量
坑数限制	坑数量 < 3	计数信号量
资源互斥	铁锹(甲乙共用)，水桶(丙独用)	互斥信号量

参与者分工

甲: 挖坑(需要铁锹)
乙: 放树苗 + 填土(填土需要铁锹)
丙: 浇水(需要水桶)

信号量设计

信号量	类型	初值	作用	使用者
empty	计数	3	控制坑数上限(<3)	甲申请，乙释放
holes	计数	0	已挖好的坑数量	甲生产，乙消费
ready	计数	0	已种好待浇水的树	乙生产，丙消费
shovel	互斥	1	铁锹互斥访问	甲乙共用

说明：水桶仅丙使用且单线程，为"尽可能少"省略 bucket 信号量

同步互斥代码实现

甲(挖坑进程)

```
甲() {
    while (true) {
        wait(empty);      // 确保坑数<3
        wait(shovel);     // 申请铁锹
        挖坑();
        signal(shovel); // 释放铁锹
        signal(holes);    // 增加可用坑数
    }
}
```

乙(种树填土进程)

```
乙() {
    while (true) {
        wait(holes);      // 等待有坑可用
        放树苗();          // 不需要工具
        wait(shovel);     // 申请铁锹填土
        填土();
        signal(shovel); // 释放铁锹
        signal(ready);    // 增加待浇水树数
        signal(empty);    // 释放坑位容量
    }
}
```

丙(浇水进程)

```
丙() {
    while (true) {
        wait(ready);      // 等待有树需浇水
        浇水();           // 使用水桶(单线程无需互斥)
    }
}
```

信号量作用机制分析

同步关系实现

同步关系	实现信号量	机制说明
甲→乙	holes	甲生产坑，乙消费坑
乙→丙	ready	乙生产待浇水树，丙消费
坑数控制	empty + holes	有界缓冲区模型

互斥关系实现

互斥资源	实现信号量	竞争进程	使用场景
铁锹	shovel	甲、乙	甲挖坑，乙填土
水桶	省略	仅丙	单线程无竞争

本方案用 **4 个信号量**实现完整同步互斥：

- **empty/holes**: 有界缓冲区控制坑数上限
- **ready**: A→B→C 流程同步
- **shovel**: 铁锹互斥访问

保证了流程正确性、资源安全性和无死锁特性。

评分标准

信号量定义（2 分）	进程同步互斥描述（3 分）	解释与说明（2 分）
完整性（1 分） 正确定义了所有必要的信号量，包括控制资源（铁锹）互斥使用的信号量和控制树坑数量的信号量，可得 2 分。 如果遗漏了控制资源互斥的信号量（如 mutex1 或 mutex2）或者遗漏了控制树坑数量的 empty 信号量，扣 1 分。 如果信号量定义完全错误，得 0 分。 **正确性（1 分）** 所定义的信号量初值正确，可得 1 分。 若有信号量初值设置错误，扣 1 分。	**进程操作完整性（2 分）** 对甲、乙、丙三个进程的操作都使用 wait()和 signal()进行了正确描述，包括对资源获取和释放的操作以及对树坑数量控制的操作，可得 3 分。 如果只正确描述了两个进程的操作，扣 1 分。 如果只正确描述了一个进程的操作，扣 2 分。 如果三个进程的操作都描述错误，得 0 分。 **操作顺序正确性（1 分）** 每个进程中的 wait()和 signal()操作顺序正确，符合逻辑，可得 1 分。 若存在操作顺序错误（例如在没有获取资源之前就释放资源等逻辑错误），扣 1 分。	**信号量作用解释（1 分）** 对所定义的每个信号量的作用都进行了准确的解释，可得 1 分。 若有信号量作用解释错误或者遗漏某个信号量的解释，扣 1 分。 **信号量初值解释（1 分）** 对信号量初值的设定依据进行了合理准确的解释，可得 1 分。 若初值解释错误或者不合理，扣 1 分。

пс.【9 分】某进程的虚拟地址空间如下图所示，阴影部分为未占用区域。



题 46 图

另现有 C 程序代码：

```
char*ptr;
void main（）{
    int length;
    ptr =（char*）malloc（size of（char））;
    scanf（"%s"，ptr）;
    length = strlen（ptr）;
    printf（"length=%din"，length）;
    free（ptr）;
}
```

请回答下列问题，并说明理由。

（1）上述程序执行时，PCB 位于哪个区域，执行 scanf（）等待键盘输入时，该进程处于什么状态？（3 分）

（2）main（）函数的代码位于哪个区域？其直接调用的哪些函数的功能需要通过执行驱动程序实现？（3 分）

（3）变量 ptr 被分配在哪个区域？若变量 length 没有被分配在寄存器中，则会被分配在哪个区域？ptr 指向的字符串位于哪个区域？（3 分）

虚拟地址空间结构

区域	特性	存储内容	访问权限
内核区	高地址，内核管理	PCB、内核数据结构	内核态访问
阴影区	未映射区域	空闲虚拟地址空间	不可访问
用户栈	向下增长	局部变量、函数参数	读写
运行时堆	向上增长	动态分配内存	读写
可读写数据区	固定大小	全局变量、静态变量	读写
只读代码区	低地址	程序指令、常量	只读可执行

(1) PCB 位置与进程状态

问题	答案	理由
PCB 位于哪个区域	内核区	PCB 是内核维护的进程控制块，属于内核数据结构
scanf 等待时进程状态	阻塞态 (I/O 等待态)	等待键盘输入，被内核挂起直到数据到达

状态转换过程：

运行态 --调用 scanf()--> 阻塞态 --键盘输入到达--> 就绪态 --调度--> 运行态

(2) 代码位置与驱动程序

函数	代码区域	是否需要驱动	底层实现
main()	只读代码区	-	用户程序代码
scanf()	库函数代码区	✓	系统调用 read + 键盘/终端驱动
strlen()	库函数代码区	✗	纯用户态内存遍历
printf()	库函数代码区	✓	系统调用 write + 显示设备驱动
malloc()/free()	库函数代码区	部分需要	用户态堆管理 + 系统调用(brk/mmap)

需要驱动程序的函数：

- **scanf()**: 键盘/终端驱动完成输入设备 I/O
- **printf()**: 显示设备驱动完成输出设备 I/O

(3) 变量存储位置分析

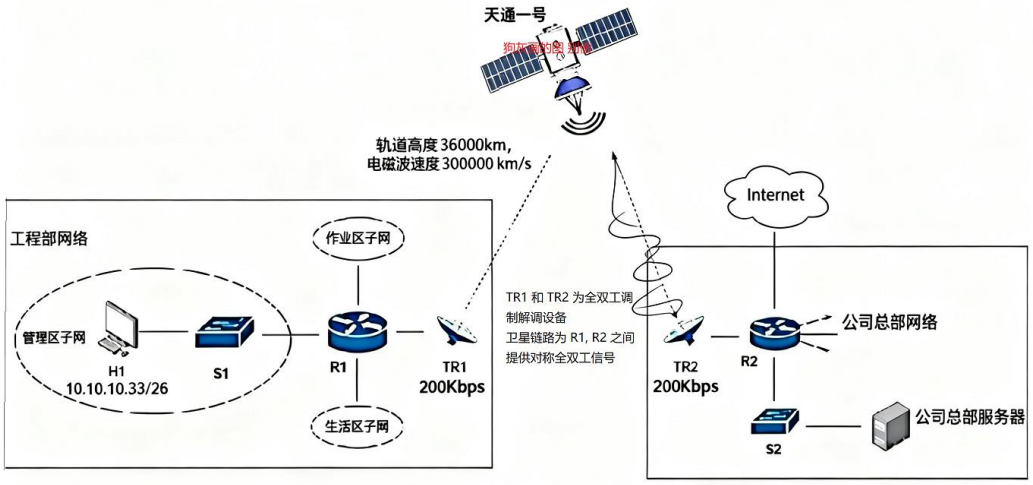
变量/数据	存储区域	分配方式	生命周期
变量 ptr	可读写数据段	启动时分配空间	贯穿整个程序
变量 length	用户栈	自动变量	main()函数执行期间
ptr 指向的字符串	运行时堆	malloc()动态分配	直到 free()释放

ptr 被分配在可读写数据段，根据虚拟地址空间划分，全局变量默认分配在可读写数据段

length 是在用户栈上分配的，因为它是在 main 函数内部定义的局部变量。

ptr 指向的字符串在运行时堆上，因为 ptr 是通过 malloc 在堆上分配内存得到的。

пт.【9 分】如题 47 图所示，卫星天通一号的轨道高度是 36000km，电磁波传播速度为 300000km/s。TR1 和 TR2 用于收发卫星信号，采用全双工调制解调，其收发速率为 200Kbps。



请回答下列问题，并说明理由。

(1)忽略卫星信号中继，TR1，TR2 调制解调开销，则 R1 到 R2 之间的卫星链路单向传播时延是多少？主机 H 向总部服务器传输数据时可达到的最大吞吐量是多少？
若忽略各层协议首部开销，以及以太网的传播时延，则 H→sever 上传一个 4000B 的文件，至少需要多长时间？（3 分）

（2）基于 GBN 为卫星链路设计单向可靠的链路层协议 SLP，支持 R1→R2 发送数据。SLP数据帧长 1500B，忽略 ACK 帧长度，要求 SLP 单向信道利用率不低于 80%，则发送窗口至少为多少？SLP 帧序号至少为多少？（3 分）

（3）总部给工程部分配 IP 地址空间 10.10.10.0/24，再划分为 3 个子网，生活区子网不少于120 个，作业子网，管理区子网 IP 均不少于 60 个，H 已正确配置 IP。问作业区子网，管理区子网，生活区子网地址各是多少？（3 分）

网络架构概述

(2)**通信方式**：工程部网络 ↔ 卫星 ↔ 公司总部网络

(3)**卫星参数**：轨道高度 36000km，电磁波速度 300000km/s

(4)**链路参数**：全双工，每方向 200Kbps 传输速率

(5)**关键设备**：TR1/TR2 调制解调器，路由器 R1/R2

（1）传播时延、吞吐量与文件传输（3 分）

卫星链路单向传播时延

参数	数值	计算过程
传输距离	72000km	地面→卫星→地面：36000×2
电磁波速度	300000km/s	题目给定
单向传播时延	**0.24s**	72000÷300000=0.24s

最大吞吐量分析

链路段	带宽限制	瓶颈分析
局域网段	通常≥100Mbps	非瓶颈
卫星链路	**200Kbps**	**瓶颈**

H→总部服务器最大吞吐量：200Kbps

4000B 文件上传时间

时间组成	计算	结果
序列化时间	4000B×8÷200Kbps = 32000÷200000	0.16s
传播时延	单向传播时延	0.24s
总传输时间	**0.16s + 0.24s**	**0.40s**

（2）GBN 协议设计参数（3 分）

基本参数计算

参数	计算过程	结果
帧发送时间 tf	1500B×8÷200Kbps = 12000÷200000	0.06s
传播时间 tprop	单向传播时延	0.24s
带宽时延积 a	tprop÷tf = 0.24÷0.06	4
RTT	2×tprop = 2×0.24	0.48s

GBN 窗口大小设计

GBN 信道利用率公式： $U = \min(1, W/(1+2a))$

窗口大小 W	利用率计算	利用率 U	是否满足≥80%
7	7÷(1+2×4) = 7÷9	77.8%	
8	**8÷(1+2×4) = 8÷9**	**88.9%**	

序号空间设计

GBN 序号要求： 避免序号混淆，需满足 序号空间 ≥ W+1

参数	要求	结果
最小序号空间	W+1 = 8+1	9
最少序号位数	**$\lceil \log_2 9 \rceil$**	**4 位**
可用序号范围	2^4	0-15（16 个）

答案：

- **发送窗口至少为：8**
- **序号位数至少为：4 位**

（3）IP 子网划分（3 分）

子网需求分析

子网	主机需求	推荐子网掩码	可用主机数
生活区	≥120	/25	126
管理区	≥60	/26	62
作业区	≥60	/26	62

VLSM 子网划分方案

地址空间： 10.10.10.0/24

子网	网络地址	子网掩码	地址范围	广播地址	可用主机
管理区	**10.10.10.0/26**	255.255.255.192	.1-.62	.63	62 个
作业区	**10.10.10.64/26**	255.255.255.192	.65-.126	.127	62 个
生活区	**10.10.10.128/25**	255.255.255.128	.129-.254	.255	126 个

最终答案总结：

题目	关键参数	结果
(1)	传播时延/吞吐量/传输时间	0.24s / 200Kbps / 0.40s
(2)	发送窗口/序号位数	8 / 4 位
(3)	管理区/作业区/生活区	.0/26 / .64/26 / .128/25